



**BERGISCHE
UNIVERSITÄT
WUPPERTAL**

Fachbereich 4

Mathematik, Informatik

Lehrstuhl für Technologien und Management
der digitalen Medien

BACHELOR-THESIS

**AUTOMATIC TEXTURE EXTRACTION ON SYNTHETIC IMAGES USING 6D
POSE ESTIMATION / AUTOMATISCHE TEXTUREXTRAKTION AUF SYN-
THETISCHEN BILDERN MITTELS 6D LAGENSCHÄTZUNG**

Oliver Jan Jarosik

1415430

Informatik

Wuppertal, den 24. April 2023

Betreuer: M.Sc. Robert Maack

Erstgutachter: Univ.-Prof. Dr.-Ing. Tobias Meisen

Zweitgutachter: Prof. Dr.-Ing. Dietmar Tutsch



BERGISCHE
UNIVERSITÄT
WUPPERTAL

Der Prüfungsausschuss für den Studiengang
Bachelor of Science Informatik

Der Vorsitzende
Univ.-Prof. Dr. Matthias Boiten

Gaußstr. 20, 42119 Wuppertal

Bergische Universität Wuppertal, Abt. 14, Zentrales Prüfungsamt,
Herr Depiereux, Gaußstr. 20, 42119 Wuppertal

Herr
Oliver Jan Jarosik
Gravelottestr. 21
42103 Wuppertal

Gesprächspartner	Herr Depiereux
Raum	T.12.17
Telefon	+49 (0)202 439-5096
Telefax	+49 (0)202 439-5269
E-Mail	depiereux@uni-wuppertal.de
Internet	zpa.uni-wuppertal.de
Aktenzeichen	3.4/PD
Datum	19.01.2023

Zulassungsbescheid Bachelor-Thesis

Ihr Antrag vom 16.01.2023

Sehr geehrter Herr Jarosik,

gemäß Ihrem o.g. Antrag, teile ich Ihnen den Titel für Ihre Bachelor-Thesis mit:

Automatische Texturextraktion auf synthetischen Bildern mittels 6D Pose Estimation

1. Prüfer/in: Herr Prof. Dr.-Ing. Meisen
2. Prüfer/in: Herr Prof. Dr.-Ing. Tutsch

Für die Bearbeitung der Master-Thesis haben Sie **3 Monate** Zeit.
Der letztmögliche Abgabetermin ist der

24. April 2023.

Die Abschlussarbeit ist in **dreifacher** Ausfertigung und **inkl. je einer CD- bzw. DVD-ROM** im ZPA einzureichen und die Erklärung gem. § 15 Abs. 6 und 7 PO 2019 beizufügen.

Im Übrigen gelten die Regelungen des § 15 der Prüfungsordnung vom 11. Juni 2019 für den Studiengang Bachelor of Science Informatik.

Mit freundlichen Grüßen
im Auftrag

P. Depiereux
Zentrales Prüfungsamt

Anlagen

Abstract

The goal of this work is to investigate the correlation of basic concepts of a 6D pose estimation approach, to the extraction of a texture. Specifically, it was intended to test how the 6-dimensional pose of an object can be of benefit to the quality of the extracted texture. For this purpose, data from 25.000 synthetic images were analyzed, where a model is trained to achieve the most accurate mapping of the texture.

Zusammenfassung

Das Ziel dieser Arbeit ist es, die Zusammenhänge von grundlegenden Konzepten eines 6D Pose Estimation Ansatzes, auf die Extraktion einer Textur zu untersuchen. Im Detail sollte getestet werden, inwieweit die 6-dimensionale Lage eines Objekts von Vorteil für die Qualität der extrahierten Textur sein kann. Dazu werden die Daten von über 25.000 synthetischen Bildern ausgewertet, bei denen ein Modell darauf trainiert wurde, ein möglichst genaues Abbild der Textur zu erreichen.

Inhaltsverzeichnis

Abbildungsverzeichnis.....	III
Tabellenverzeichnis.....	III
Abkürzungsverzeichnis	III
1. Einleitung.....	1
1.1 Motivation.....	1
1.2 Aufbau der Arbeit	2
2. Grundlagen.....	3
2.1 Definition und Abgrenzung	3
2.1.2 RANSAC (Random Sample Consensus)	5
2.1.3 PnP (Perspective n Point).....	7
2.2 Related Work	8
2.2.1.1 You only look once (YOLO)	9
2.2.1.2 Real-Time Seamless Single Shot 6D Object Pose Prediction.....	10
2.2.1.3 PVNet (Pixel-wise voting network).....	13
2.2.2 Datensätze	14
2.2.2.1 Hinterstoisser et al (LINEMOD).....	15
2.2.2.2 T-LESS Datensatz.....	16
2.2.3.1 Segment Anything Meta	16
2.2.3.2 BundleSDF Nvidia.....	17
3 Methodik.....	19
3.1 Zielsetzung	19
3.2 Generierung der synthetischen Daten	19
3.3 Aufbau der Datengenerierung.....	20
3.4 Programmdesign	23
3.5 Datenaufbereitung	25
4. Evaluation.....	26
4.1 Ergebnisse	26
4.2 Diskussion.....	27

4.4.1 Limitation des Modells	29
4.4.2 Implikationen	30
4.5 Ausblick	30
5. Literaturverzeichnis.....	31
6. Zusätzliches Material.....	36

Abbildungsverzeichnis

Abbildung 1: Referenzmarker [Bos16]	4
Abbildung 2: SSD-Netzwerk [KMT+17].	4
Abbildung 3: RANSAC [FB81].....	6
Abbildung 4: Perspective-n-Point [FB81]	8
Abbildung 5: Metriken [Tom13], [Ivi22]	13
Abbildung 6: LINEMOD [HLI+13]	15
Abbildung 7: Datensatz.....	21

Tabellenverzeichnis

Tabelle 1: Labels.....	22
Tabelle 2: Ergebnisse	26

Abkürzungsverzeichnis

6DoF	-	Six Degree of Freedom
RANSAC	-	Random Sample Consensus
YOLO	-	You Only Look One
IoU	-	Intersection over Union
CNN	-	Convolutional Neural Network
PnP	-	Perspective-n-Point
RGB	-	Red Green Blue
RGB-D	-	Red Green Blue – Depth

1. Einleitung

Die Lagenschätzung ist ein Teilgebiet der Computer Vision, mit der Aufgabe die Verschiebung und Drehung von starren Objekten relativ zum Koordinatensystem der Kamera zu bestimmen. Während der aktuelle Stand der Technik in einigen Testdatensätze eine sehr gute Leistung erzielt, gibt es immer noch eine Reihe von Bereichen, für die diese Ansätze unzureichend sind [TSF18]. Der Umgang mit Verdeckungen und Veränderungen des Aussehens war der Schwerpunkt der jüngsten Forschung [HHO+17], doch ein Faktor, der die Verbreitung dieser Algorithmen stark einschränkt, ist ihre Fähigkeit, sich leicht und effizient an neue, ungesehene Objekte anzupassen.

Die derzeit führenden Ansätze zur Schätzung der Pose Estimation nutzen Machine Learning und neuronale Netze zur Lokalisierung des Zielobjekts in einer Szene und zur Bereitstellung einer groben Schätzung der Lage des Objekts.

Die Lage eines Objekts wird als 6-dimensionaler Vektor dargestellt, der die (x,y,z) Koordinaten des Objekts und die Drehung des Objekts abbilden. Um das Feld von der menschlichen Pose Estimation, welche die Position von menschlichen Körperteilen voraussagt, abzugrenzen, kann die Aufgabe auch als 6D Pose Estimation bezeichnet werden, um explizit die auf die Schätzung der 6 Parameter, die sowohl die Position als auch die Orientierung bestimmen, zu verdeutlichen. In dieser Arbeit wird die visuelle Lagenschätzung mittels Algorithmen thematisiert, die allein auf visuellen Informationen aus einem RGB-Bild basieren, ohne dass zusätzliche Objektmarkierungen oder Sensoren erforderlich sind.

1.1 Motivation

6D Pose Estimation ermöglicht es, die Lage und Orientierung eines Objekts im Raum zu bestimmen. Durch präzise Bestimmung dieser Werte, könnten unerwünschte Verzerrungen minimiert und die Qualität einer Textur dadurch verbessert werden. Dieses Gebiet hat auch zukünftig großes Potential, da es in vielen Bereichen angewendet werden könnte. Zum Beispiel der Robotik, um Objekte zu identifizieren und zu greifen, in der Augmented Reality, um realistischere Texturen und Umgebungen zu erzeugen oder der Architektur und im Design, um Materialien und Texturen genauer zu reproduzieren. Eine automatische Texturextraktion mittels 6D Pose Estimation könnte eine spannende Möglichkeit sein, die Qualität von Texturen zu verbessern und in verschiedenen Bereichen Anwendung zu finden.

1.2 Aufbau der Arbeit

Um ein grundlegendes Verständnis für den Aufbau des Programms zu bekommen, wird zunächst ein Überblick über 6D Pose Estimation gegeben. Dabei werden sowohl Grundlagen, als auch unterschiedlichen Methoden kurz erläutert. Zusätzlich werden zwei erprobte Modelle kurz charakterisiert sowie das in dieser Arbeit benutzte Modell genauer beschrieben. Um die Bewertung dieser Modelle zu ermöglichen, werden zwei bekannte Datensätze charakterisiert. Dies bietet auch eine Hilfestellung und ein Verständnis für die Generierung des neuen Datensatzes. Zusätzlich werden zwei 2023 erschienene Modelle beschrieben, welche den konzeptionellen Rahmen abschließen. Der nachfolgende Teil wird die Generierung des synthetischen Datensatzes näher beleuchten. Anschließend folgt die theoretische Ausarbeitung einer Bewertungsmetrik sowie ihre Integration. Zum Schluss werden die Ergebnisse aufgearbeitet und im Zusammenhang mit den bestehenden Erkenntnissen diskutiert.

2. Grundlagen

2.1 Definition und Abgrenzung

Die 6D Pose Estimation, auch bekannt als Lagenschätzung in **sechs Freiheitsgraden** (6DOF), bezieht sich auf das Problem der Schätzung, der Position und der Orientierung eines Objekts in einer 3-dimensionalen Umgebung. Dies kann in einer Vielzahl von Anwendungen eingesetzt werden, z. B. im Bereich von **Computer Vision**, der Augmented Reality und der Robotik.

Bei der 6D Pose Estimation besteht das Ziel darin, die **3D-Position** eines Objekts (x, y, z) sowie seine **Ausrichtung** (Roll, Pitch, Yaw) zu bestimmen, welche für die Rotation um die drei lot-rechten Achsen, die Rotation um die Längsachse, die Querachse und die Hochachse, stehen. Zur Darstellung dieser werden häufig Euler-Winkel oder Rotationsmatrizen genutzt.

Die Verfahren zur 6D Pose Estimation lassen sich in drei Hauptkategorien einteilen: **Referenzmarker basierte Pose Estimation**, **Vorlagen basierte Pose Estimation** und **visuelle Pose Estimation mittels Machine Learning**.

Die Referenzmarker basierte Pose Estimation beruht auf der Idee, dass es ein definiertes, einheitliches **Symbol** auf dem Bild existiert. Mithilfe dieses Symbols, lassen sich die Entfernung, die Position und andere Metriken des Objekts erkennen. Vorteil dieses Systems ist, dass man robuste Ergebnisse erhält. Auch Lichtverhältnisse spielen keine große Rolle, da schwarz- und weiße Symbole einen hohen Kontrast besitzen. Nachteile bietet dieser Ansatz dabei, dass man nicht jedes Objekt mit solch einem Referenzmarker bestücken kann. Würde man beispielsweise alle Objekte in einem Warenhaus damit markieren, wäre der Aufwand unglaublich hoch. Deshalb ist diese Lagenbestimmung theoretisch sehr ausgereift, jedoch in der Praxis oder in der Realität oft nicht nutzbar (Abbildung 1).

Vorlagenbasierte Pose Estimation benutzt neben einem RGB-Bild auch eine **Tiefenkarte** des Szenarios. Diese kann beispielsweise mit RGB-D Kameras aufgenommen werden. Diese Art der Lagenbestimmung wird für gängige Aufgaben immer seltener verwendet. Hintergrund ist, dass die Forschung sich darauf spezialisiert, aus einem Bild die Lage zu entnehmen, ohne zusätzliche Informationen wie die Tiefe. „Multimodel templates for real-time detection of textureless object in heavily cluttered scenes“ [vgl. HHC+11] ist dabei ein bekannter Algorithmus, welches den geläufigen LINEMOD Datensatz nutzt [HLI+12]. Dabei werden die Bilder sowie die Tiefe einer Szene extrahiert und mit Vorlagen verglichen, welche das Objekt aus verschiedenen Winkeln scannt. Der Abgleich aller Vorlagen mit der Vorhersage ist sehr zeitaufwändig, weshalb dieser Ansatz der Pose Estimation größtenteils durch die Pose Estimation mittels Machine Learning ersetzt wird.

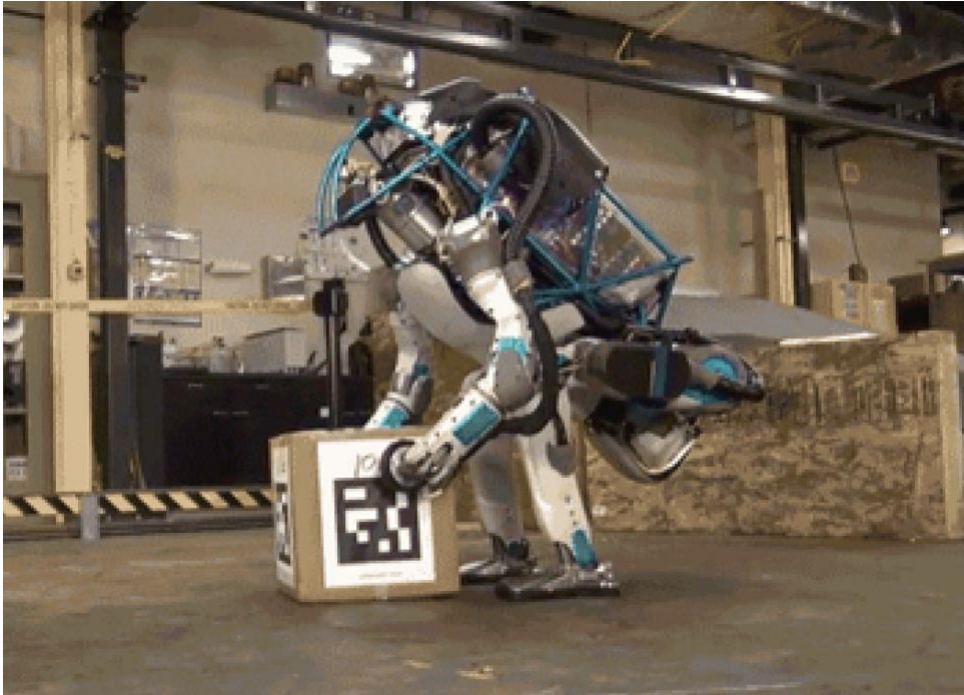


Abbildung 1 Referenzmarker: Boston Dynamics
Der Roboter kann Anhand der Referenzmarker das Objekt greifen und bewegen.

Durch die rasante Entwicklung und Verbreitung des **Machine Learning** Bereichs, wurden auch die Techniken in Relation zum Anwendungsfall der Pose Estimation weiterentwickelt. Diese Algorithmen arbeiten ohne Markierungen oder Sensoren, sondern erkennen die Lage des Objektes direkt aus dem Bild heraus und sind dabei zusätzlich robust gegenüber Okklusion. Der in der Thesis verwendete Algorithmus [TSF18] kommt ebenfalls aus diesem Teilbereich. Ebenfalls wird in Kapitel 2.2.4 das 6D Pose Estimation Modell PVNet [PLH19+] genauer beschrieben.

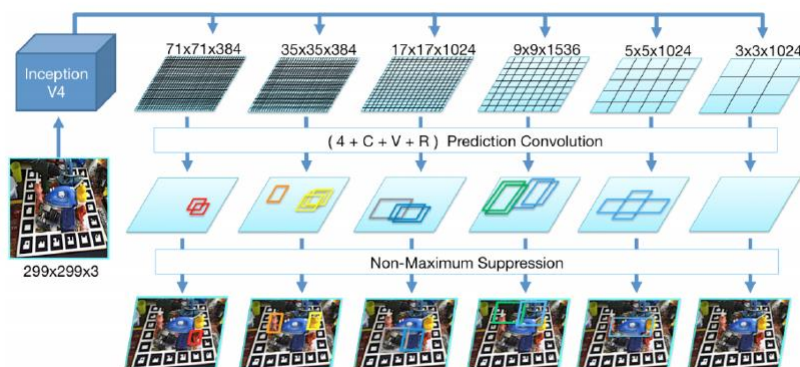


Abbildung 2 SSD-Netzwerk: Struktur eines SSD-6D Netzwerkes
C beschreibt die Anzahl der Objektklassen, V die Blickwinkel und R die Rotationen.

Trotz allem ist ein schwerwiegendes Problem der 6D Pose Estimation, das Vorhandensein von **Rauschen und Unsicherheit** in Daten, welche aufgrund der hohen Anzahl von Freiheitsgraden gegeben ist. Diese entstehen, da selbst geringfügige Änderungen im Winkel der Lage, alle anderen Freiheitsgrade deutlich verändern können. Dabei gibt es zwei grundlegende Konzepte, welche Helfen diese zu bereinigen.

2.1.2 RANSAC (Random Sample Consensus)

RANSAC oder "RANdom Sample Consensus", ist eine **iterative Methode** zur **Schätzung** der Parameter eines mathematischen Modells aus einer Reihe von Beobachtungsdaten, die durch die Existenz von **Ausreißern** sonst zu fehlerhaften Prognosen führen kann. Dieser Algorithmus wurde im Jahre 1978 von M.A. Fischler und R.C. Bolles entwickelt [FB81].

Es ist eine weit verbreitete Methode auf dem Gebiet der Computer Vision und der Bildverarbeitung und hat auch in anderen Bereichen wie der Robotik und der Geodatenanalyse Anwendung gefunden [HW11], [CTL+15], [RFM23].

Eines der Hauptprobleme, die RANSAC lösen soll, ist die **Auslöschung von Ausreißern** in den beobachteten Daten. Ausreißer sind Datenpunkte, die nicht dem Muster entsprechen, welche von der Mehrheit der Daten beschrieben wird. Wenn diese im Anpassungsprozess miteinbezogen werden, kann dies die Genauigkeit eines Modells erheblich beeinträchtigen. Indem nur eine Teilmenge der Daten zur Anpassung des Modells verwendet wird, kann RANSAC die Auswirkungen von Ausreißern auf das Endergebnis abschwächen. Vergleichbare ältere Algorithmen wie die Methode der kleinsten Quadrate haben keinen Mechanismus um diese Fehler/Ausreißer zu erkennen. Dort wird von guten Daten ausgegangen.

Die Grundidee von RANSAC besteht darin, eine kleine Teilmenge der beobachteten Datenpunkte nach dem **Zufallsprinzip** auszuwählen, um sie zur Anpassung eines Modells zu verwenden. Diese Teilmenge wird verwendet, um Vorhersagen für die verbleibenden Datenpunkte zu treffen und die Anzahl der Ausreißer zu identifizieren. Liegt die Anzahl der Ausreißer über einem bestimmten Schwellenwert, wird das Modell als gut geeignet für die Daten angesehen und für Vorhersagen verwendet. Liegt die Anzahl der Ausreißer unter dem Schwellenwert, wird der Vorgang mit einer neuen Teilmenge von Datenpunkten iterativ wiederholt, bis ein zufriedenstellendes Modell gefunden ist.

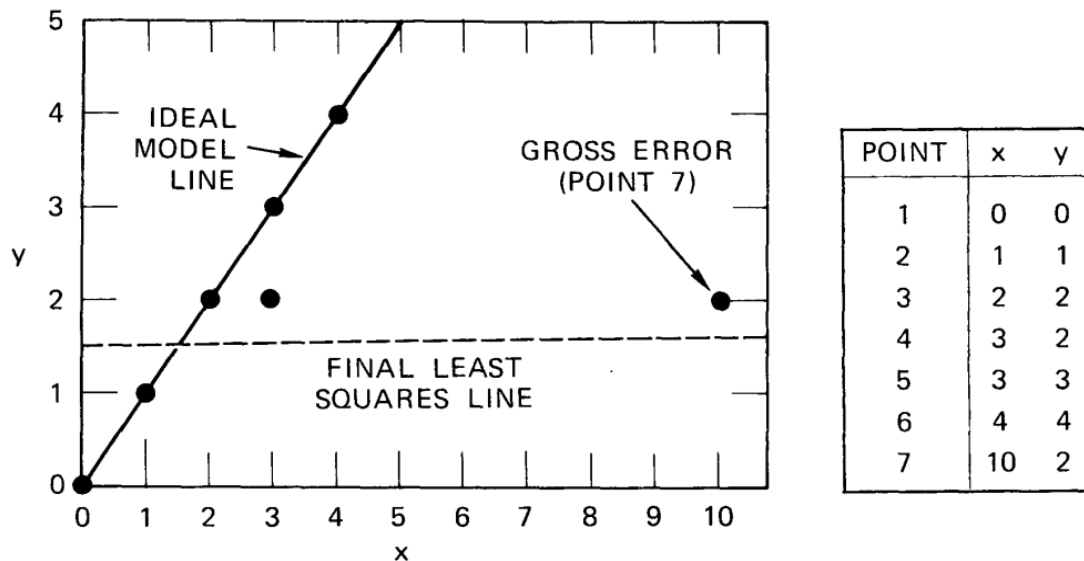


Abbildung 3 RANSAC: Beispiel eines RANSAC Algorithmus
 Man sieht, dass die Linie des least square Algorithmus aufgrund des Punktes 7 deutlich von der idealen Linie abweicht.

Um ein Verständnis für den Algorithmus zu erhalten, ist hier eine grundlegende Formulierung eines RANSAC Algorithmus.

$$1 - p = (1 - u^m)^N \quad (1)$$

$$\Leftrightarrow N = \frac{(\log(1-p))}{\log(1-(1-v)^m)} \quad (2)$$

N gibt die Anzahl der Iterationen an und p die Wahrscheinlichkeit, dass kein Ausreißer vorhanden ist. Dabei repräsentiert u die Wahrscheinlichkeit, dass der Datenpunkt ein Inlier ist, also kein Ausreißer. Das Gegenteil dessen wird als $v = 1 - u$ bezeichnet, also die Wahrscheinlichkeit, dass ein Ausreißer auftritt. Die mindestens benötigten Punkte werden in m angegeben [Der10]. Dieser lässt sich ebenfalls durch weitere Eigenschaften erweitern wie beispielsweise durch Nutzen einer *Maximum Likelihood* Schätzung [TD00] oder eines *Importance Samplings* [TZ00]. Zusammenfassend lässt sich sagen, dass RANSAC ein leistungsfähiges Werkzeug für die Schätzung der Parameter eines mathematischen Modells aus einem Satz von Beobachtungsdaten ist und sich besonders für die Behandlung des Problems der Ausreißer eignet. Es funktioniert, indem es iterativ Modelle an Teilmengen der Daten anpasst und das Modell mit den meisten Ausreißern auswählt, so dass es die Parameter des Modells auch bei Rauschen und Ausreißern robust schätzen kann.

Im Bereich der 6D Pose Estimation wird RANSAC dazu verwendet, die richtigen Pixel auf einem Bild auszuwählen. Ein 480 x 640 Pixel großes Bild bietet 307.200 Pixel. Das gesuchte Objekt besitzt aber nur einen Bruchteil davon. Pixel welche **nicht zum Objekt** gehören, werden dabei Mithilfe von RANSAC aussortiert und Rauschen im Modell minimiert.

2.1.3 PnP (Perspective n Point)

Perspective-n-Point Algorithmen (PnP-Algorithmen) sind eine Klasse von Verfahren zur **Pose Estimation** (d.h. der Lage und Orientierung) eines Objekts im 3D-Raum anhand einer Reihe von 2D-3D-Punktkorrespondenzen. Diese Korrespondenzen werden zwischen einer Reihe von bekannten 3D-Punkten auf dem Objekt und ihren entsprechenden 2D-Projektionen in einem von einer Kamera aufgenommenen Bild hergestellt. Das Ziel von PnP-Algorithmen ist es, die Lage des Objekts im Koordinatensystem der Kamera auf der Grundlage dieser Korrespondenzen zu bestimmen.

PnP-Algorithmen sind in einer Vielzahl von Computer-Vision-Aufgaben nützlich, einschließlich Augmented Reality [DKB14], Robotik [SSHB12] und Objekterkennung [QSMG16]. Insbesondere werden sie häufig bei der 6D Pose Estimation verwendet, bei der die 3D-Position und 3D-Ausrichtung eines Objekts in einer Szene geschätzt werden muss. Diese stellt ein anspruchsvolles Problem dar, da nicht nur die 3D-Position des Objekts genau geschätzt werden muss, sondern auch seine Ausrichtung, die sich schon bei kleinen Bewegungen des Objekts oder der Kamera erhebliche Veränderung aufzeigt. PnP-Algorithmen können diese Herausforderungen meistern, indem sie Optimierungsverfahren einsetzen, um die Pose zu finden, die am besten zu den 2D-3D-Korrespondenzen passt.

Es gibt mehrere verschiedene PnP-Algorithmen, die für die 6D-Lagenschätzung verwendet werden können, jeder mit seinen eigenen Stärken und Schwächen. Einige bekannte Beispiele sind der ICP-Algorithmus [Bai22], der Levenberg-Marquardt-Algorithmus [Lev44] und der bereits erwähnte RANSAC-Algorithmus. Insgesamt ist die Verwendung von PnP-Algorithmen für die 6D Pose Estimation wichtig, da sie eine zuverlässige und effiziente Methode, sowohl von der Laufzeit, als auch von der Komplexität, zur Schätzung der Lage eines Objekts im 3D-Raum auf der Grundlage seiner 2D-Projektionen in einem Bild bietet.

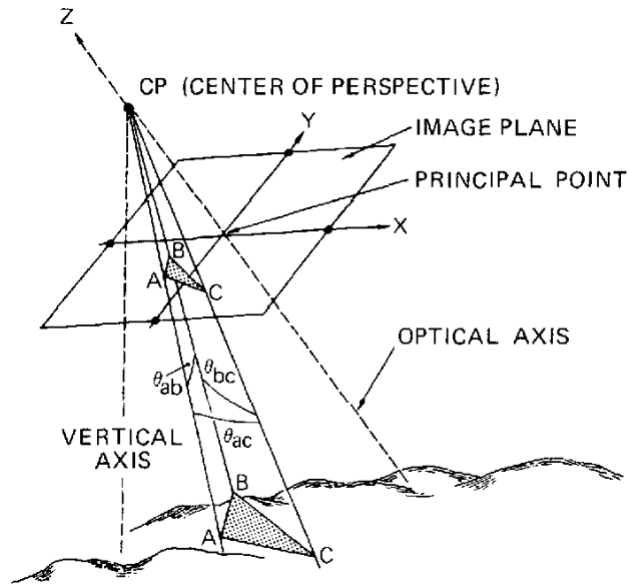


Abbildung 4 Perspective-n-Point: Geometrie des Standortbestimmungsproblems
Die Bestimmung einer Menge von m Kontrollpunkten, deren dreidimensionale Koordinaten (A, B, C) in einem Koordinatensystem bekannt sind.

Mit diesen beiden Grundlagen ist der Grundbaustein für komplexere Modelle gelegt, welche häufig in der 6D Pose Estimation Verwendung finden.

2.2 Related Work

6D Pose Estimation besitzt mittlerweile eine **Vielzahl** von Modellen, welche sich über die Jahre teilweise sehr verändert haben. Um einen grundlegenden Überblick zu schaffen, werden im Folgenden drei Modelle kurz erläutert, um eine Vorstellung davon zu schaffen, wie sich diese über die Jahre verändert haben. Dabei wird ein **älteres** YOLO (you only look once) Modell beschrieben, da dieses für einige 6D Pose Estimation Ansätze als Voraussetzung dient. Das zweite Modell wurde in Zusammenarbeit mit Microsoft erforscht und ist Grundlage für viele weitere zukünftige Ansätze. Dieser Ansatz wird in Kapitel 3 wiederverwendet. Das dritte Modell enthält einen sehr interessanten, wenn auch rechenaufwendigen, Ansatz welcher eine grobe Idee gibt, wie sich zukünftige Modelle entwickeln könnten.

2.2.1.1 You only look once (YOLO)

You Only Look Once (YOLO) ist ein Deep-Learning-Algorithmus, der für die **Objekterkennung** in der Computer Vision erforscht wurde. YOLO wurde von Joseph Redmon et al. an der University of Washington entwickelt und 2016 vorgestellt [RDGF16]. Seitdem hat er sich zu einem beliebten Algorithmus für die Echtzeit-Objekterkennung in Bildern und Videos entwickelt.

YOLO ist ein **einstufiger Objekterkennungsalgorithmus**, der ein Convolutional Neural Network (CNN) verwendet, um gleichzeitig die Bounding Box und Klassenwahrscheinlichkeiten für ein bestimmtes Bild vorherzusagen. Im Gegensatz zu anderen Objekterkennungsalgorithmen, die mehrere Durchgänge durch das Bild oder die Verwendung von Region Proposal Networks [RHGS16] erfordern, verarbeitet YOLO das komplette Bild einmalig und sagt die Bounding Box sowie die Klassenwahrscheinlichkeiten unmittelbar voraus. Dies geschieht schnell und effizient, so dass es in Echtzeit sowie auf Geräten mit geringem Stromverbrauch laufen kann.

Der YOLO-Algorithmus besteht aus mehreren Convolutional Layers, gefolgt von einigen Fully-connected Layern. Die Convolutional Layers werden verwendet, um Merkmale aus dem Eingabebild zu extrahieren, während die Fully-connected (linear) Layers zur Vorhersage der Klassenwahrscheinlichkeiten und Bounding Boxes für die Objekte im Bild verwendet werden. YOLO unterteilt das Eingabebild in ein Gitter von **Zellen** und sagt die **jeweiligen Wahrscheinlichkeiten** voraus. Jede Zelle ist für die Erkennung einer bestimmten Anzahl von Objekten zuständig. Die Anzahl der Zellen, die zur Unterteilung des Bildes verwendet werden, wird durch die Größe des Eingabebildes und die Größe der zu erkennenden Objekte bestimmt.

YOLO verwendet eine Technik namens **Anchor Boxes** (Ankerboxen), um die Genauigkeit der Objekterkennung zu verbessern. Dies sind vordefinierte Begrenzungsboxen unterschiedlicher Größe und Seitenverhältnisse, die dem Algorithmus helfen, die Begrenzungsboxen für die Objekte im Bild vorherzusagen. Die Ankerboxen basieren auf der Verteilung der Objektgrößen im Trainingsdatensatz. YOLO verwendet eine Technik namens K-Means Clustering zur Bestimmung der Ankerboxen.

Der YOLO-Algorithmus hat mehrere Vorteile gegenüber anderen Objekterkennungsalgorithmen. Zum einen ist es die **Schnelligkeit**, zum anderen ist er in der Lage, **mehrere Objekte** in einem einzigen Durchgang zu erkennen, was ihn effizienter macht als Algorithmen, die mehrere Durchgänge durch das Bild erfordern. Darüber hinaus besitzt YOLO die Möglichkeit, Objekte

in verschiedenen Größen und Ausrichtungen zu erkennen, was es **robuster** als Modelle macht, die auf eine feste Anzahl beschränkt sind.

Trotz seiner Vorteile hat YOLO einige Einschränkungen, wie beispielsweise bei der Erkennung kleiner oder teilweise verdeckter Objekte. Eine weitere Einschränkung ist, dass es in bestimmten Situationen zu falsch-positiven oder falsch-negativen Ergebnissen kommen kann. Mittlerweile ist YOLO weniger präzise als andere Objekterkennungsalgorithmen, die komplexere Architekturen oder Ensemblemodelle verwenden.

2.2.1.2 Real-Time Seamless Single Shot 6D Object Pose Prediction

Das Model Single Shot Deep CNN wurde im Dezember 2018 von Bugra Tekin, Sudipta N. Sinha und Pascal Fua veröffentlicht [TSF18]. Dieser Single-Shot-Ansatz nutzt ein RGB-Bild um eine **Vorhersage der 6D Lage** eines Objektes zu treffen. Grundlage des Modells ist eine CNN Basis, welche in Echtzeit die 2D Bildpositionen der 3D Bounding Box des Objektes vorhersagt [RF16]. Infolgedessen wird damit sowie mit Hilfe eines PnP-Alorithmus die 6D-Lage des Objekts geschätzt. Dieses Modell wurde sowohl auf den LINEMOD Datensatz, als auch auf dem OCCLUSION Datensatz trainiert und verfeinert. In dieser Thesis wird ein eigener Datensatz erstellt, welcher mit diesem Modell verwendet wird. Mehr dazu in Kapitel 3.2.

Dabei wird ein Objekt in **9 Knotenpunkte** parametrisiert, acht davon für die acht Punkte der Bounding Box und der letzte Knotenpunkt für den Zentroiden.

Die Ausgangszielwerte werden als Tensor der Größe $S \times S \times D$ gespeichert. Dabei wird $i \in S \times S$ bestimmt, um die räumliche Position des Tensors in Form eines D-dimensionalen Vektors v_i wiederzugeben. Um zu bestimmen, in **welchem Gebiet** sich das Objekt befindet, wird zusätzlich zum YOLO IoU (intersection over union) Ansatz eine Funktion genutzt (siehe Formel (3)).

$$c(x) = \begin{cases} e^{\alpha \left(1 - \frac{D_T(x)}{d_{th}}\right)}, & \text{wenn } D_T(x) < d_{th} \\ 0 & \text{sonst} \end{cases} \quad (3)$$

Diese Funktion gibt einen Wert $c(x)$ an, welcher die Distanz $D_T(x)$, die als euklidische Distanz gemessen zum Wahrheitswert, berechnet. Dabei wird der Wert d_{th} genutzt, um die exponentielle Funktion abzuschneiden. Das α gibt dabei die Krümmung der Funktion an.

Die **Netzwerkstruktur** beruht auf einer YOLO v2 Architektur [RF16] und besteht aus 23 Convolutional Layers und fünf Max-Pooling Layers. Der Wert für S wird dabei auf 13 gesetzt, um ein 13 x 13 2-dimensionales Feld zu erzeugen, auf welchem die Vorhersagen getroffen werden. Der letzte Layer trifft dabei Voraussagen über die (x, y) Koordinaten der Kontrollpunkte. Formel 3 wird dazu genutzt um die **Distanz** zwischen dem geschätzten Wert und der Ground Truth zu berechnen. Der Zentroid wird als Schwerpunkt der jeweiligen Zelle gesetzt. Hierbei sind (c_x, c_y) , die 2D- Koordinaten der oberen linken Ecke der zugehörigen Gitterzelle, in welcher der Zentroid liegen muss. Die **vorhergesagten Punkte** (g_x, g_y) werden dabei, mit f als Sigmoid Funktion, wie folgt definiert:

$$g_x = f(x) + c_x \quad (4)$$

$$g_y = f(y) + c_y \quad (5)$$

Mithilfe dieser Funktionen wird die richtige Zelle des Objekts ermittelt. Um diesen Prozess weiter zu optimieren wird eine **Verlustfunktion** aufgestellt, welche mit den Termen $\mathcal{L}_{pt}$, $\mathcal{L}_{conf}$, $\mathcal{L}_{id}$ die Koordinaten, die Konfidenz und den Klassifizierungsverlust beschreiben. Dabei wird ein Mean-Squared Error mit dem Wert $\lambda_{conf} = 0.1$ für die Koordinaten, Konfidenzen mit dem Wert $\lambda_{pt} = 1$ gewichtet sowie eine Cross Entropy Funktion für den Klassifizierungsverlust auf $\lambda_{id} = 1$ gesetzt.

$$\mathcal{L} = \lambda_{pt}\mathcal{L}_{pt} + \lambda_{conf}\mathcal{L}_{conf} + \lambda_{id}\mathcal{L}_{id} \quad (6)$$

Damit kann man ein **2D-Rechteck** definieren, welches das Objekt und seine unmittelbare Umgebung maskiert. Außerdem hat man die örtliche Umgebung des Objekts bestimmt und kann mit der Pose Estimation fortfahren.

Die Lage des Objekts wird einmalig bestimmt durch den klassenspezifischen Konfidenzwert. Das neuronale Netzwerk gibt also die 2-dimensionale Projektion der 3-dimensionalen Hüllkörper an. Mithilfe dieser neun korrespondierenden Punkte wird mittels des in 2.1.3 erwähnten PnP Algorithmus die Transformations- sowie die Rotationsmatrix des Objekts in Bezug zur Kamera berechnet. Somit erhalten wir die 6D Lage des gesuchten Objekts.

Zur Evaluation ist eine Metrik notwendig die beschreibt, wie gut das Modell vorhersagen kann. Singleshopose nutzt dabei drei Metriken, welche auch im Bereich der 6D Pose Estimation häufig Verwendung finden.

Der **Reprojection Error** (Projektionsfehler) sieht eine Schätzung als gut an, falls die durchschnittliche Distanz zwischen der 2-dimensionalen Projektion der 3-dimensionalen Mesh Eckpunkte und den Wahrheitswerten weniger als 5 Pixel beträgt. Dies ist besonders in Bereichen der erweiterten Realität (Augmented Reality) ausschlaggebend, da man die tatsächliche Projektion mit den Wahrheitswerten vergleicht (Abbildung 5, links).

Die nächste Metrik ist die **Intersection over Union (IoU)** Methode, in welcher eine Maske des 3D Objekts sowohl über Wahrheitswerte, als auch über die berechnete Pose überlappt werden. Dabei wird die Pose als korrekt angesehen, wenn die Überlappung größer als 0,5 ist (Abbildung 5, rechts).

Die dritte Metrik wird als **ADD-Metrik** bezeichnet. Dabei werden die durchschnittlichen Distanzen der geschätzten 3D Koordinaten sowie die Wahrheitswerte verglichen. Falls diese kleiner als 10 % des Durchmessers des Objekts sind, wird die Lage als korrekt angesehen. Da es bei symmetrischen Objekten aufgrund von weniger prägnanten Orientierungspunkten zu Ausreißern in der Schätzung kommen kann, wird bei diesen Objekten ebenfalls die Rotations- sowie die Transformationsmatrix miteinbezogen, welche mit denen der Wahrheitswerten verglichen werden.

Es ist wichtig zu erwähnen, dass dieses Modell keine perfekten Ergebnisse erzielt. Für die spätere Forschung sollte dies in Betracht gezogen werden. Die durchschnittliche 3D Distanz (mean ADD) beim LINEMOD Datensatz beträgt gerade mal 55,9 % Genauigkeit. Daher sollte die Erwartungshaltung für die eigene Forschung basierend auf diesem Modell nicht zu hoch sein.

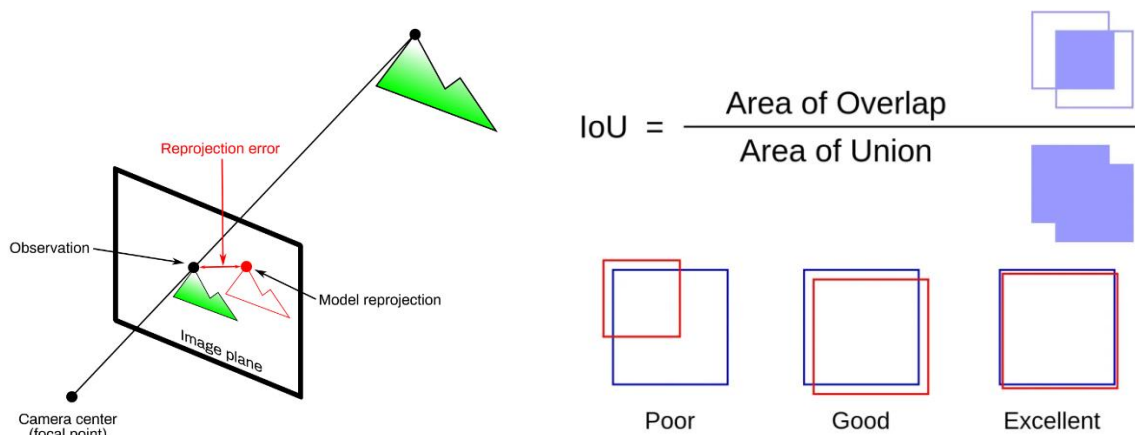


Abbildung 5 Metriken: (Links: Ein Beispiel für einen Reprojection Error. Rechts: Beispiel für ein IoU Fehler)

2.2.1.3 PVNet (Pixel-wise voting network)

PVNet ist ein Algorithmus, der von Sida Peng, Yuan Liu, Qixing Huang, Hujun Bao und XiaoWei Zhou im Jahre 2018 entwickelt wurde und auf dem Prinzip der **Punkt-Voxel-Zuordnung** beruht [PLH+19]. Die Grundidee besteht darin, eine **Kombination** aus 2D-Bildmerkmalen und 3D-Punktwolkendaten zu verwenden, um die Lage eines Objekts zu schätzen. Der Algorithmus verwendet ein tiefes neuronales Netz, um die 2D-Bildmerkmale auf die 3D-Punktwolkendaten abzubilden. Dadurch kann der Algorithmus die Lage eines Objekts durch den Vergleich der 3D-Punktwolkendaten mit den Bildmerkmalen voraussagen.

Der PVNet-Algorithmus verwendet einen zweistufigen Ansatz für die 6D-Lagenschätzung. In der ersten Stufe erzeugt der Algorithmus eine Reihe von **2D-Schlüsselpunkten** auf dem Objekt von Interesse. Diese Schlüsselstellen werden anhand ihrer charakteristischen Merkmale, wie Ecken oder Kanten, ausgewählt. Anschließend bildet der Algorithmus diese Schlüsselstellen mit Hilfe einer Punkt-Voxel-Mapping-Technik auf eine 3D-Punktwolke ab.

In der zweiten Phase verwendet der Algorithmus die 3D-Punktwolke, um die Position des Objekts im 3-dimensionalen Raum zu schätzen. Dies geschieht indem die Ausrichtung und Position des Objekts auf der Grundlage der 3D-Punktwolkendaten **vorhergesagt** wird. Das neuronale Netz verwendet eine Reihe von Merkmalen, um relevante Informationen aus der 3D-Punktwolke zu extrahieren und eine Reihe von 3D-Koordinaten zu erzeugen, die der Ausrichtung und Position des Objekts entsprechen. Die Punktwolke wird dabei aus **Vektoren** definiert und die Bewegung und Drehung des Objekts somit bestimmt.

Eines der wichtigsten Merkmale des PVNet-Algorithmus ist seine Fähigkeit, mit Verdeckungen und Unschärfen im Eingangsbild umzugehen. Der Algorithmus ist so konzipiert, dass er

irrelevante Merkmale im Bild erkennt, herausfiltert und sich auf die **markantesten konzentriert**. Dadurch kann der Algorithmus die Position eines Objekts auch dann schätzen, wenn es teilweise verdeckt oder von anderen Objekten umgeben ist. Dadurch ist er sehr **geeignet gegenüber Rauschen**.

Ein weiteres Hauptmerkmal des PVNet-Modells ist seine Effizienz. Der Algorithmus ist so konzipiert, dass er **rechnerisch effizient** ist und in Echtzeit auf einem Standard-Desktop-Computer ausgeführt werden kann. Dadurch eignet er sich ideal für Anwendungen, die eine schnelle und genaue 6D-Lagenschätzung erfordern, wie z. B. Robotik oder autonomes Fahren.

2.2.2 Datensätze

Ein wichtiger Bestandteil um 6D Pose Estimation Modelle **bewerten** zu können, sind die Datensätze, auf welchen sie schlussendlich trainiert werden. Dabei haben sie viele Abweichungen untereinander, sei es die Qualität der Bilder, die Genauigkeit der Modelle oder Rauschen bzw. Überdeckungen innerhalb der Daten. Um ein grundsätzliches Verständnis dafür zu bekommen, wie Datensätze für 6D Pose Estimation aussehen, werden anbei zwei bekannte dargestellt, welche für **State of the Art** 6D Pose Estimation genutzt und als Grundlage für Bewertungen herangezogen werden.

Insgesamt sind für die Arbeiten an 6D Pose Estimation Modellen eine Reihe von verschiedenen Datensätzen erstellt worden. Die elementaren Teile solcher Datensätze bestehen einerseits aus einer Reihe von Bildern, die das Zielobjekt enthalten, andererseits aus einer Reihe von Ground Truth Labeln, welche die **Transformation** zwischen dem Zielobjekt im Bild und dem Kamerabild angeben. Da das Zielobjekt mit keinem intrinsischen Koordinatensystem assoziiert wird, wird das Koordinatensystem durch die **Transformationsmatrix** und die **Rotationsmatrix** bestimmt. Da sich im Laufe der Zeit sowohl die Robustheit, als auch die Genauigkeit dieser Algorithmen zur Lagenschätzung verbessern, werden neue Datensätze entwickelt, welche neben komplexeren und anspruchsvolleren Eigenschaften auch gezielt auf Problemstellungen aus bestehenden Daten eingehen. Die große Schwierigkeit bei der 6D Pose Estimation besteht darin, den **exakten** Wert für die Matrixtransformation zwischen Kamera und Zielobjekt zu bestimmen. Im Folgenden werden zwei weit verbreitete Datensätze kurz vorgestellt sowie deren Methoden zur Bestimmung der Ground Truth dargelegt.

2.2.2.1 Hinterstoisser et al (LINEMOD)

Im Jahre 2013 erschien einer der wichtigsten Datensätze im Bereich der 6D Pose Estimation von Hinterstoisser et al [HLI+12], welcher besser bekannt ist als **LINEMOD-Datensatz** und nicht mit dem LINEMOD-Algorithmus verwechselt werden sollte. Insgesamt umfasst der Datensatz 15 unterschiedliche Haushaltsobjekte mit über 1000 Bildern jedes einzelnen Objekts, die sich durch Form und Farbe unterscheiden. Zusätzlich sind diese grundsätzlich ohne Textur und besitzen eine starre Struktur. Bei den Bildern werden die Objekte unübersichtlich mittig auf einem Tisch platziert, sodass Überlappungen und Verdeckungen der einzelnen Objekte entstehen können. Diese Bilder wurden mit einer Kinect-RGBD-Kamera aufgenommen, welche neben den RGB-Bildern auch noch die **Tiefenkarten** der Bilder enthält. Um eine gute Verteilung zu schaffen, wurden die Aufnahmen in einer Art Hemisphäre gleichmäßig, aber zufällig über dem Tisch erfasst (Abbildung 6, links). Dies gewährleistet unterschiedliche Posen und Winkel der Kamera in Relation zum Objekt. Referenzmarkierungen wurden am Rand des Tisches platziert, um die Pose der Kamera relativ zur Welt zu bestimmen. Dies erlaubt uns eine Transformation zwischen den Weltkoordinaten und dem Koordinatensystem des Zielobjekts abzuleiten. Zusätzlich wird jedes einzelne Objekt als 3D-Modell beigefügt, jedoch ohne Präzisierung zu Ihrer Generierung oder Zusammenstellung (siehe Abbildung 6, rechts).

Ein mögliches Problem des Datensatzes könnte im Gebrauch von neuronalen Netzen liegen. So besteht die Möglichkeit, dass ein neuronales Netz nicht versucht instinktiv die Pose des Objekts zu lernen, sondern das Zielobjekt relativ in Position der Referenzmarker bestimmt.

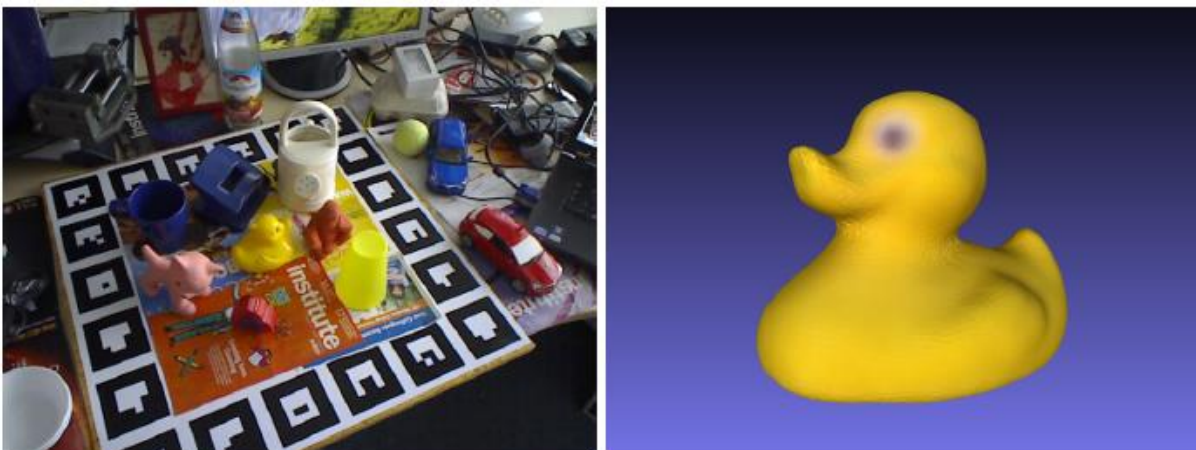


Abbildung 6 LINEMOD (Links: Ein Beispielbild aus dem LINEMOD Datensatz mit den Objekten in der Mitte des Tisches und den Markierungen umkreisend. Rechts: Ein 3D-Modell eines Objekts, in dem Fall der Ente.)

2.2.2.2 T-LESS Datensatz

Der T-LESS Datensatz wurde von Hodan et al. [HHO+17] im Jahre 2017 entwickelt, besteht aus etwa 50.000 Bildern, die 30 verschiedene elektronischen Verbrauchskomponenten enthalten: Diese sind zum Großteil aus **weißem Kunststoff** und neben mehreren Symmetrieachsen ist auch wenig Farbvariation vorhanden. Aufgrund der geringeren Unterschiede innerhalb der Objekte ist dies ein besonders anspruchsvoller Datensatz für die Bestimmung der Lage. Die Trainings- und Testdatensätze werden dabei klar unterschieden. Während der Trainingsdatensatz aus einzelnen Objekten auf schwarzem Hintergrund besteht, wird der Testdatensatz dadurch hervorgehoben, dass er, ähnlich wie der LINEMOD Datensatz, Bilder mit mehreren Objekten auf einem Tisch beinhaltet. Auch der Hintergrund, die Anzahl der Objekte sowie der Schärfegrad des Bildes differiert.

Der Datensatz wurde sowohl mit RGB- als auch mit RGBD-Kameras in einem bestimmten Bereich von Scheitelkreis und Höhenlage aufgenommen. Der Scheitelkreis wird mit einer Drehscheibe simuliert, während die Höhenlage der Kamera auf zehn Schritte eingestellt ist. Daraus folgen insgesamt 1.926 Trainingsbilder für jedes Objekt, welche sich auf der oberen Hemisphäre verteilen.

Die 3D-Modelle, die in T-LESS bereitgestellt werden unterscheiden sich in zwei Arten. Eines besteht aus 3D-Modellen, die manuell erstellt wurden, die anderen sind halbautomatisch generierte Modelle, welche mithilfe des Trainingsdatensatzes erzeugt wurden.

Im Unterschied zum Hintertoisser-Datensatz werden im Trainingsdatensatz keine Referenzmarker verwendet. Dies bringt den Vorteil, dass das Modell nicht auf die Referenzmarker, sondern das Objekt trainiert wird. Im Testdatensatz sind diese jedoch vorhanden, sodass zur Probe eine Kamera-zu-Welt Transformation berechnet werden kann.

Zusammenfassend lässt sich der T-LESS Datensatz als ein Datensatz mit hochwertigen Bildern und akkuraten Posen, jedoch mit zeitaufwendiger Generierung beschreiben.

2.2.3.1 Segment Anything Meta

Meta Platforms, Inc. hat kürzlich ein eigenes Programm namens „SAM“ (Segment Anything Model) zur Bildsegmentierung vorgestellt, das im Jahr 2023 von Alexander Kirillov et al. veröffentlicht wurde [KMR+23]. SAM verfügt über ein eigenes Modell und einen neuen Datensatz für die Bildzerlegung (Image Segmentation). Ähnlich wie YOLO erkennt SAM Objekte und erstellt Masken um ihre Umrisse, um sie von anderen Teilen des Bildes abzugrenzen. Eine

Besonderheit dabei ist, dass das Modell auf über 11 Millionen lizenzierten Bildern trainiert wurde und dadurch über 1 Million Masken generiert hat.

SAM gehört zur Kategorie der sogenannten „Foundation Models“, also großer Modelle, die auf großen Mengen von ungelabelten Daten trainiert werden und anschließend für spezifische Aufgaben angepasst werden. Im Bereich Computer Vision gab es bereits solche Modelle, wie z.B. ALIGN [JYX+21] oder DALL-E [RPG+21], jedoch werden diese oft in Kombination von Text- und Bildmodalitäten genutzt. Segment Anything hingegen legt den Grundstein für die Bildsegmentierung.

Das Modell besteht aus drei Teilen: Einem Bildkodierer, einem Eingabeaufforderungskodierer und einem Maskendekodierer. Der Bildkodierer teilt das Bild in passende Bildteile auf, um auch hochauflösende Bilder verarbeiten zu können. Der Eingabeaufforderungskodierer ermöglicht die Auswahl zwischen „sparse“ spärlichen Eingabeaufforderungen, die aus Punkten, Boxen und Texten bestehen, und „dense“ (dichten) Eingabeaufforderungen, die aus den Masken bestehen. Die Masken werden mittels Convolution-Operationen gefiltert und dann elementweise zur Gesamtmaske summiert. Der Maskendekodierer gibt zusätzlich ein Ausgabetoken aus, das eine dynamische Maskenbildung gewährleistet und auch zur Vorhersage des Maskenorts genutzt wird. Da SA nur eine einzige Ausgabe hat, kann es vorkommen, dass Objekten mehrere Masken zugeordnet werden. Hierbei wird die maximale Anzahl auf drei Maskenausgaben begrenzt und mittels eines Confidence-Scores die richtige Maske bestimmt, wobei der IoU (Intersection over Union) -Wert für jede Maske berechnet wird.

Ein weiterer Vorteil des Modells ist seine hohe Effizienz, da es mit geringer CPU-Belastung im Webbrowser läuft und somit eine nahtlose Echtzeiteingabeaufforderung ermöglicht. Insgesamt bietet SA ein grundlegendes Framework für zukünftige Herausforderungen im Bereich Computer Vision, beispielsweise für präzise Texturerkennung mit dem Modell.

2.2.3.2 BundleSDF Nvidia

Wen et al. [WTB+23] stellen mit BundleSDF eine Methode zur Verfolgung eines unbekannten Objekts aus einer monokularen RGBD-Videosequenz in nahezu Echtzeit (10Hz) vor, während gleichzeitig eine neuronale 3D-Rekonstruktion des Objekts durchgeführt wird. Die Methode funktioniert für beliebige starre Objekte, auch wenn keine visuelle Textur vorhanden ist. Es wird davon ausgegangen, dass das Objekt nur im ersten Bild segmentiert ist. Es werden keine zusätzlichen Informationen benötigt sowie keine Annahmen über den Interaktionsagenten getroffen. Der Schlüssel zu dieser Methode ist ein neuronales Objektfeld, das gleichzeitig mit

einem Optimierungsprozess des Lagegraphen erlernt wird, um Informationen in einer konsistenten 3D-Darstellung zu akkumulieren, die sowohl Geometrie als auch Aussehen erfasst. Zur Erleichterung der Kommunikation zwischen den Threads wird automatisch ein dynamischer Pool von Pose-Memory-Frames verwaltet. Ihr Ansatz bewältigt anspruchsvolle Sequenzen mit großen Lagenänderungen, teilweiser und vollständiger Okklusion, unstrukturierten Oberflächen und Glanzlichtern.

Zu den Beiträgen der Autoren gehört eine neuartige Methode zur kausalen 6-DoF-Positionsverfolgung und 3D-Rekonstruktion eines neuartigen, unbekannten dynamischen Objekts, die ein Co-Design von gleichzeitigen Verfolgungs- und neuronalen Rekonstruktionsprozessen nutzt. Diese laufen nahezu in Echtzeit.

3 Methodik

3.1 Zielsetzung

Kann die **Qualität von einer Texturextraktion** aus einem Bild dahingehend verbessert werden, dass man sich die Position des Objekts, durch eine Bestimmung der 6D Lage, zu Nutzen macht? Welchen Einfluss kann diese Manipulation auf den Zustand der Textur haben und welche Probleme können dabei entstehen? Ziel ist es, diese Zusammenhänge mit Hilfe von gesammelten Daten, auf verschiedenen Objekten, festzustellen, bei denen eine Eckpunkte der Bounding Box extrahiert und mit denen der zu extrahierten idealen Textur in einer Metrik abgeglichen werden. Besonderes Augenmerk liegt hierbei auf der Automatisierung des Prozesses.

3.2 Generierung der synthetischen Daten

Synthetische Daten haben einige Vorteile gegenüber Daten, welche händisch generiert werden. Mit synthetischen Daten hat man die volle **Kontrolle** über die Art, den Umfang und die Verteilung der generierten Daten. Dies ermöglicht eine hohe Flexibilität bei der Gestaltung von Testszenarien, um spezifische Anwendungsfälle oder Randbedingungen abzudecken und die Leistung von Algorithmen oder Modellen zu bewerten. Ebenfalls können diese so angepasst werden, dass gezielt Fehlerfälle oder Anomalien simuliert werden, um die Qualität von Modellen oder Algorithmen zu überprüfen und zu verbessern. So sind in Programmen, mit welchen man synthetische Daten erstellen kann, üblicherweise integrierte Koordinatensysteme verfügbar, welche es erleichtern die Wahrheitswerte aus den Daten zu erhalten, die so in der realen Welt nicht definiert. Zusätzlich können synthetische Daten schnell und einfach generiert werden, ohne aufwendige Datenerhebungen oder Datenbereinigungen durchführen zu müssen. Dies ermöglicht eine hohe **Skalierbarkeit** bei der Entwicklung, insbesondere in Umgebungen, in denen große Datenmengen erforderlich sind.

Jedoch sollte darauf geachtet werden, da sich die Datenqualität im Vergleich realen Daten unterscheiden kann. So kann es sein, dass die Übertragbarkeit von Modellen auf echte Daten nicht gewährleistet ist, weshalb die Nutzung stets mit Ergebnissen der realen Welt abgeglichen und diskutiert werden muss.

3.3 Aufbau der Datengenerierung

Es gibt es eine Vielzahl von 3D-Modellierungsprogrammen, welche je nach Anwendungsgebiet unterschiedliche Vorteile bieten, wie beispielsweise die Anbindung von Animationssoftwares oder die Integration einer Programmierschnittstelle. Sowohl Autodesk Maya als auch **Blender** sind zwei beliebte Softwareprogramme, die in der 3D-Modellierung und Animation weit verbreitet sind.

Autodesk Maya ist eine professionelle Software, welche in der Unterhaltungsbranche viel Anklang findet, da es eine breite Palette von Funktionen bietet. Jedoch wird bei Autodesk Maya eine Lizenz benötigt, was einen Unterschied zum kostenlosen Ansatz von Blender darlegt. Blender hingegen ist eine **Open-Source-Software** die von jedem geändert und individuell angepasst werden kann. Es basiert auf den Programmiersprachen C, C++ und Python, wodurch es eine API-Schnittstelle zu Python bietet, welche zur Erstellung von Skripten genutzt werden kann. Zwar bietet Blender weniger Plugins als Maya, trotzdem gibt es einige exzellente Erweiterungen wie Blenderproc, welche an Machine Learning Probleme angepasst sind.

Blenderproc [DWS+23] ist eine auf Python basierte Erweiterung für Blender. Es kann eine Szene in Blender auf die Tiefe sowie auf die semantische Unterteilung der Objekte generieren. Ebenfalls ist dadurch eine zufällige Rotation des Objekts sowie der Farbe möglich. In dieser Arbeit wird Blender verwendet, da die Open-Source Natur für mögliche weitere Forschung eine bessere Kompatibilität bietet. Auch die Python Schnittstelle ermöglicht eine gute Automatisierung des Prozesses.

Um einen Datensatz für diese Arbeit zu generieren, ist es von großer Wichtigkeit, dass das zu untersuchende Objekt sorgfältig ausgewählt wird. In diesem Fall wurden Klemmbausteine als Grundlage für den Datensatz gewählt, die auf unterschiedliche Weise angeordnet und zusammengesteckt werden können. Dies bietet die Möglichkeit, eine Vielzahl von Variationen zu generieren und so eine große Anzahl an Variation von Szenarien für die Forschung zu schaffen. Die Automatisierung des Prozesses ermöglicht es, schnell und einfach mehrere Datensätze zu generieren, was die Skalierbarkeit des Forschungsprojekts erheblich verbessert. Darüber hinaus wurde im Vorfeld festgelegt, welche Variationen ausgetestet werden sollen, um potenzielle Grenzen oder Schwachstellen zu identifizieren und das Verständnis für das Modell und seine Funktionsweise zu vertiefen.

Die Erstellung von Datensätzen für die automatisierte Erkennung und Klassifizierung von Objekten ist ein entscheidender Schritt bei der Entwicklung von Algorithmen und Modellen. In diesem Zusammenhang wird in dieser Arbeit die Blender Python API genutzt, um einen Datensatz bestehend aus etwa 1250 Bildern zu generieren. Hierbei wird eine Kamera in Hoch-

Achse zum Objekt platziert, während sich das Objekt um seine eigene Achse dreht. Der Zentroid des Objekts befindet sich stets im Mittelpunkt des Koordinatensystems. Ein Beispiel ist in Abbildung 7 dargestellt.

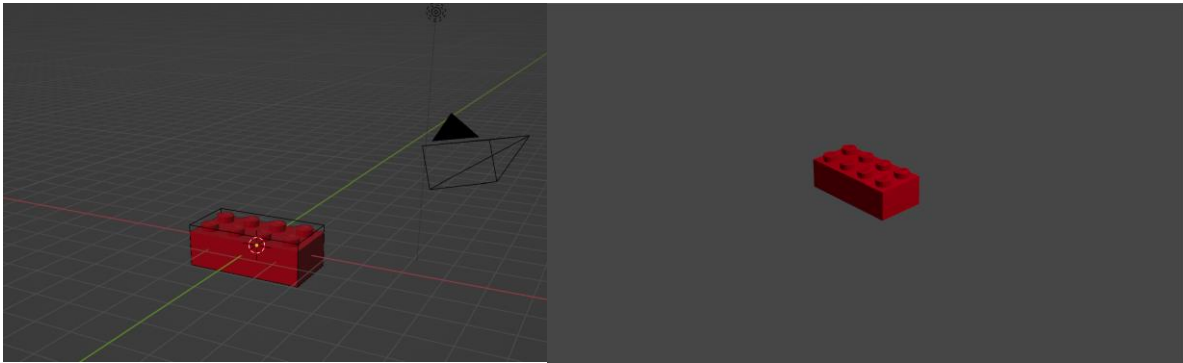


Abbildung 7 Datensatz: (Links: Aufbau innerhalb von Blender. Rechts: Gerendertes Bild des Objekts)

Um die Generierung des Datensatzes effizient zu gestalten, wird eine for-Schleife verwendet. Dabei werden wichtige Erkenntnisse über das Objekt und dessen Eigenschaften gewonnen, die bei der Entwicklung von Algorithmen und Modellen zur automatisierten Erkennung und Klassifizierung von Objekten hilfreich sind. Ein Beispiel dafür sind die einzelnen Punkte der Bounding Box, welche in einer separaten Textdatei abgespeichert werden.

Während der Generierung des Datensatzes werden die Ground-Truth-Daten für das Modell berechnet. Diese sind neben der acht Ecken der Bounding Box auch der Mittelpunkt des Objekts. Der Zentroid des Objekts befindet sich stets an den Schnittpunkten der Koordinatenachsen. Blender berechnet eine Bounding Box für das Objekt und markiert die 3-dimensionalen Eckpunkte, welche die x-, y-, z-Koordinaten des Punktes enthalten. Diese Eckpunkte werden mithilfe der Kamerakoordinaten und einer Transformationsmatrix in 2-dimensionale Punkte transformiert, die auf das entstandene Bild des Objekts abgestimmt sind. Dadurch entstehen 640x480 große PNG-Dateien sowie Label-Textdateien, die von 0 bis 1235 nummeriert sind und 21 Dezimalzahlen enthalten. Die führende Null repräsentiert die Klasse des Objekts. Die nächsten zwei Dezimalzahlen geben die Koordinaten des Zentroiden des Objekts an, wobei als Referenz der untere linke Rand des Bildes dient. Dabei wird vom unteren linken Rand ausgegangen und die Zahl als Differenz zwischen der maximalen Höhe und Breite des Bildes errechnet. Die folgenden 16 Dezimalzahlen sind die jeweiligen x- und y-Koordinaten der einzelnen Eckpunkte der Bounding Box. Die letzten beiden Dezimalzahlen geben die Breite und Höhe der Bounding Box im Vergleich zum Bild an, welche als Maximum der Datenpunkte berechnet

wird. Alle Dezimalzahlen werden wie bei einer Sigmoid Funktion zwischen null und eins angegeben. Ein Beispiel dieser Labels ist in Tabelle 1 zu sehen.

ID	Class	x- und y Koordinaten des Zentroiden	8 Mal x- und y Koordinaten der Bounding Box	Höhe	Breite
1	0	(0.5, 0.5)	8 x (0.3, 0.3)	0.6	0.3
2	0	(0.5, 0.5)	8 x (0.3, 0.3)	0.6	0.4
3	0	(0.5, 0.5)	8 x (0.3, 0.3)	..	..
4	...	...	...	...	...

Tabelle 1 Labels: Beispiel der Labels mit neun Keypoints

Die PNG Dateien sind notwendig, um Masken zu generieren, welche den Output des Modells verbessern. Durch die Nutzung von Masken wird das Objekt auf einen zufälligen Hintergrund trainiert und somit die Stabilität verbessert. Nach dem Erstellen der Masken, werden die PNG Dateien in JPEG umgewandelt, dies ist effizienter und verringert den genutzten Speicherplatz. Das 3D-Modell des Objekts muss im Polygon File Format abgespeichert werden. Die Laufzeit des Modells kann durch die Anzahl der Vertices im 3D-Modell stark beeinträchtigt werden. So werden Objekte mit wenigen Eckpunkten deutlicher schneller geladen und aktiviert als etwaige, welche aus mehr Vertices bestehen. Bei Tests, in denen ein Legobaustein in sehr viele einzelne Vertices aufgeteilt wurde, ist die Laufzeit signifikant angestiegen. Dies hat zur Folge, dass sehr komplizierte Objekte dazu führen könnten, dass sich die 6D-Pose Estimation stark verlangsamt. Eine Besonderheit Blenders ist die Auflistung der Koordinatenachsen. Während nahezu alle Programme mit kartesischen Koordinatenachsen arbeiten, ist Blenders Koordinatensystem anders codiert. Die y-Achse ist negativ zu jener im kartesischen Koordinatensystem. Dies hat zur Folge, dass Objekte in Blender die an der Y-Achse nach oben orientieren, in anderen Programmen nach unten orientieren.

Der Datensatz besteht aus elf Objekten. Die ersten beiden Objekte sind gewöhnliche 4 x 2 Legobausteine in Rot. Davon weist der erste Datensatz eine gute Belichtung auf, während der zweite Datensatz schlecht belichtet ist. Dies führt dazu, dass die Ecken und Kanten des Würfels schwerer für das menschliche Auge, zu identifizieren sind. Die nächsten drei Datensätze sind Kombinationen von 2 x 2, 2 x 4 und 2 x 8 Legosteinen. Diese sind so zusammengefügt, dass daraus eine Art Würfel entsteht, bestehend nur aus den Wänden ohne einen inneren Teil. Die Drei Datensätze unterscheiden sich farblich, haben aber die gleiche Form. Der Datensatz dieser Gruppe ist abwechselnd grün und blau gestapelt, wodurch Linien in der jeweiligen

Farbe entstehen. Die Farbe der Gruppe des zweiten Datensatzes wurde zufällig generiert und ist bunt. Der dritte Datensatz dieser Gruppe besteht vollständig aus roten Steinen. Die nächsten beiden Datensätze bestehen aus sieben 2×4 Legobausteinen, bei denen die unterste Ebene aus drei Bausteinen, die mittlere aus zwei und die oberste wieder aus drei Bausteinen besteht. Dadurch entstehen Löcher, welche textur- und objektlos sind. Diese sind ebenfalls in einem Datensatz mit guter Belichtung und einem Datensatz mit schlechter Belichtung aufgeteilt. Die nächste Gruppe von Legosteinen ist in Form einer Mauer angeordnet, wobei die äußeren Steine eine andere Farbe haben als die Steine in der Mitte. Ein Datensatz hat einen gelben Rand und einen schwarzen Innenteil, während der andere einen blauen Innenteil und einen roten Rand besitzt. Zum Schluss wurden noch zwei Datensätze verwendet, welche eine Ähnlichkeit mit einer Pyramidenform aufweisen. Dadurch ist der obere Teil der Bounding Box sehr weit entfernt von den Vertices der generierten Legobausteine. Diese sind ebenfalls einmal mit guter, einmal mit schlechter Belichtung aufgenommen. Beispiele dieser Objekte sind in Kapitel 6 zu sehen.

3.4 Programmdesign

Das Hauptprogramm basiert auf dem 6D Pose Estimation Modell von Tekin et al. [TSF18], welche auf einige Arten modifiziert wurde, um die gewünschten Ergebnisse zu analysieren. Wichtig ist zu erwähnen, dass das Modell auf zwei unterschiedlichen Komponenten aufbaut, einerseits auf dem Training der Daten, andererseits auf der Validierung der Daten. So wird das Modell mit den selbst erstellten Datensätzen eingebunden und trainiert, um anschließend im Validierungsprozess in Echtzeit die extrahierte Textur sowie die Ergebnisse der Bewertungsmetriken zu erhalten. Das Programm basiert auf Python, sowie auf den bekannten Bibliotheken PyTorch, Numpy, SciPy, Pillow und OpenCV.

Der Trainingsprozess verwendet ein CNN, um die prägnanten Punkte des Bildes zu erkennen und die Bounding Box mit Hilfe von Labels vorherzusagen. Als Eingabe wird ein Bild der Größe 640×480 Pixel verwendet. Der erste Layer des CNN ist ein Tensor mit den Abmessungen $416 \times 416 \times 3$, wobei jeder Wert eine Zuordnung zu den RGB-Farbwerten im Bereich von 0 bis 255 hat. Dies ermöglicht die Verwendung von Bildern mit ihren Farbwerten als Eingabe. Das neuronale Netz besteht aus insgesamt 32 Layern davon 24 Convolutional Layers, fünf Max Pooling Layers, zwei Route Layers, einem Reorg Layer und einem Detection Layer. Der Ausgang des Netzwerks ist normalerweise ein Tensor mit den Abmessungen $13 \times 13 \times (18 + 1 + C)$, wobei 13×13 die Größe des Feldes beschreibt, in welche das Bild aufgeteilt wurde, 18 für

die Position der 9 x 2 x- und y Werte der Knotenpunkte, 1 für die Confidence Value und C für die Klassenwahrscheinlichkeiten. Für dieses Modell wurde der Ausgang auf 13 x 13 x (8 + 1 + C) geändert, da für die Extraktion der Textur nur die vier Eckpunkte des Objekts relevant sind, anstatt der zuvor definierten acht Punkten der Bounding Box sowie des Zentroiden. Entsprechend müssen auch die Labels angepasst werden.

Die Evaluierung des Forschungsobjekts unterscheidet sich ebenfalls von dem ursprünglichen Design von Tekin et al. [TSF18]. Statt der ursprünglichen drei Bewertungsmetriken Reprojection Error, IoU Score und ADD Metrik wird nur die die ADD Metrik beibehalten, da die anderen beiden auf der Überlagerung des 3D-Modells basieren. Diese nutzen nämlich die Rotations- und Transformationsmatrizen, welche bei einer Umstellung auf vier Keypoints den Umfang dieser Thesis sprengen würde. Zusätzlich wird eine eigene Metrik definiert, die je nach Position des vorhergesagten Eckpunkts unterschiedlich bewertet wird. Jeder der vier Eckpunkte wird mit 25 % der Gesamtbewertung gewichtet. Je nachdem, ob die Vorhersage innerhalb oder außerhalb des Trapezes liegt, welches die vier Ground Truth Keypoints festlegen, wird die Schwere des Fehlers unterschiedlich bewertet. Fehler außerhalb des Trapezes werden nicht so schwerwiegend bewertet wie Fehler innerhalb, da das Fehlen von Texturinformationen zu größeren Problemen bei der weiteren Nutzung der Textur führen kann. Nach außen hin wird der Fehler linear bewertet, mit einer maximalen Differenz von sechs Pixeln. Bei Fehlern innerhalb der Ground Truth wird ein quadratischer Fehler angesetzt, der durch eine quadratische Funktion definiert ist. Dadurch wird ein kleiner Fehler weniger stark bewertet als ein Fehler, der tief in die Textur eingreift. Der maximale Fehler beträgt dabei eine Differenz von vier Pixeln. Es ist wichtig zu erwähnen, dass diese Bewertungsmetrik je nach Größe des Objekts im Bild angepasst werden sollte, da kleinere Objekte automatisch zu kleinerem Fehler führen würden, da die Distanzen zwischen den einzelnen Eckpunkten kleiner sind (vgl. Kapitel 6, Programmcode 1).

Zusätzlich zur Evaluierung des Modells wird auch die Textur extrahiert. Im Validierungsprozess werden die einzelnen Eckpunkte berechnet und basierend darauf die Textur extrahiert. Hierbei wird ein Trapez um die Eckpunkte gezogen und die Textur wird durch den maximalen und minimalen Wert der x- und y-Koordinaten definiert. Diese vier Punkte werden dann mit einer weißen Linie verbunden, um den gewünschten Bereich der Textur zu erhalten. Dies hat den Vorteil, dass man die extrahierte Textur im Vergleich zum Objekt analysieren und mögliche Eigenschaften besser erkennen kann. Anschließend wird mithilfe von OpenCV diese Maske ausgelagert (Beispiele in Kapitel 6).

Dieser Ansatz wird anschließend mit dem bereits definierten Ansatz verglichen, um eine Bewertung und Diskussion über die Qualität der Veränderung durchzuführen.

3.5 Datenaufbereitung

Um einen Fokus auf die Texturextraktion zu legen, wurde nur eine Rotation von maximal 90° gewählt. Falls die Rotation auf eine zufällige Zahl gesetzt wird, verändern sich die gewählten Punkte der Bounding Box auf den Hintergrund. Dies führt dazu, dass die extrahierte Textur willkürlich erscheint und keine einheitliche Messbarkeit herrscht. So führte die Texturextraktion auf dem LINEMOD Datensatz zu sehr unbefriedigenden Ergebnissen.

Die Generierung der Masken wird mit einem Python Skript erstellt, welches die Bilder in Graustufen unterteilt und ab einem bestimmten Schwellenwert den Bereich entweder Schwarz oder Weiß färbt. Für Objekte mit dunklen oder schwarzen Bereichen könnte die Erstellung der Masken Fehler, welche sich negativ auf das Training der Daten auswirken, produzieren. Auch sollten Werte, welche eine Metrik mit 100 % versieht, kritisch bewertet werden, da es wahrscheinlich ist, dass es sich hier, um einen Fehler bei der Berechnung des Wertes handelt.

4. Evaluation

4.1 Ergebnisse

Durch eine Zusammenführung der Ergebnisse aller Datensätze, die zu unterschiedlichen Gewichtungen trainiert und analysiert worden waren, ergibt sich folgende Tabelle:

Singleshopose					
	Category	ADD-10 %	5px 2D Proj.	Metric 9key	Metric 4key
Group 1	LegoBrick 1_1	26.1%	100%	63.4%	61.0%
	LegoBrick 1_2	6.2%	100%	73.8%	32.8%
Group 2	LegoMix 1 BlGr	37.6%	20.8%	60.2%	36.9%
	LegoMix 1 Color	10.4%	76.2%	52.8%	56.0%
	LegoMix 1 Red	0.0%	63.0%	37.8%	38.9%
Group 3	LegoMix 2_1	12.9%	100%	72.5%	55.3%
	LegoMix 2_2	50.9%	90.1%	69.8%	25.4%
Group 4	LegoMix 3 BlYe	41.0%	100%	48.8%	13.6%
	LegoMix 3 ReBl	4.8%	100%	63.9%	37.9%
Group 5	LegoMix 4_1	0.0%	100%	62.1%	54.0%
	LegoMix 4_2	0.0%	100%	63.5%	67.1%

Tabelle 2 Ergebnisse: Prozentzahlen geben die Genauigkeit des Modells für den jeweiligen Datensatz an

Die Datensätze wurden in fünf Gruppen unterteilt, welche jeweils dasselbe 3D-Modell verwenden. Unterschiede zwischen den Datensätzen bestehen in der Belichtung (z.B. „gut belichtet“ vs. „schlecht belichtet“) oder den spezifizierten Farben. Der Tabelle lässt sich entnehmen, dass das ursprüngliche Modell, welches auf neun Keypoints trainiert, im Allgemeinen zu besseren Ergebnissen führt, also das neue Modell, welches lediglich auf vier Keypoints trainiert.

Die verschiedenen Metriken sind in der oberen Zeile definiert. Die „Add-10 %“ Metrik gibt die Genauigkeit der Add-Metrik mit einem maximalen Threshold von 10 % an. Dies bedeutet, dass eine Pose mit einer Verschiebung von bis zu 10 % der maximal zulässige Fehler ist. Die Spalte „5px 2D Proj.“ gibt den Projektionsfehler an, der eine maximale Distanz von fünf Pixeln zwischen den 2D-Projektionen aller Punkte erlaubt. Die beiden nachfolgenden Metriken sind in Kapitel 3.4 definiert und sind einmal auf neun Keypoints und einmal auf vier Keypoints evaluiert. Dabei ist die maximale Distanz nach Innen auf vier Pixel begrenzt und nach außen auf sechs.

Die Werte sind in Prozent angegeben, wobei 0 % für eine schlechte Vorhersage der Punkte steht und 100 % für eine perfekte. In der Spalte des Projection Errors sind viele Werte mit 100 % vorhanden, die auf einen Fehler in der Berechnung hinweisen und daher für die Auswertung nicht berücksichtigt werden können.

Bei Betrachtung der Unterschiede zwischen den Keypoint Metriken fällt auf, dass sechs von elf Modellen bessere Ergebnisse erzielen, wenn sie auf neun Keypoints statt der vier Keypoints trainiert werden. Bei den anderen fünf Modellen sind die Ergebnisse ähnlich und weichen um wenige Prozentpunkte voneinander ab. Mögliche Gründe dafür könnten sein, dass ein neuronales Netz versucht, prägnante Punkte im Bild zu merken, wie Formen oder typische Merkmale. So haben Objekte, die aus mehr Vertices bestehen, einen höheren Wiedererkennungswert. Dadurch hat ein Netz, welches versucht, neun Stellen eines Objektes zu merken, mehr Bezugspunkte als eines, das nur auf vieren trainiert wird.

Bis auf zwei Ausnahmen zeigen Objekte, welche mit besseren Lichtverhältnissen trainiert und validiert wurden bessere Ergebnisse als jene mit schlechteren Lichtverhältnissen. Modelle die versuchen, die Bounding Box von bunten Objekten zu prognostizieren, zeigen generell bessere Ergebnisse, als Objekte, welche eine einzige Farbe besitzen. So hat der bunte Legomix 2 in allen Metriken sehr gute Werte. Auch hat dieser den höchsten Prozentwert, sowohl in der ADD-Metrik, als auch dem Projection Error erhalten. Die eigenen Metriken erzielten dabei leicht andere Ergebnisse. So erzielte Legobrick 1_2 den besten Wert in dem auf neun Keypoints trainierten Modell, während das auf vier Keypoint trainierte Modell überraschenderweise Legomix 4_2 am besten vorhersagen kann, obwohl die Form für Bounding Box Vorhersagen schwierig erscheint. Dies ist insofern überraschend, als dass die Bounding Box weit entfernt zu den Vertices der Pyramide liegen. Auch ist spricht eine schwache Belichtung für wenig Kontrast und daher für weniger Bezugspunkte. Trotzdem hatte diese Gewichtung den Höchstwert.

Insgesamt sieht man ein paar interessante Zusammenhänge, jedoch müsste man für fundiertere Ergebnisse weitere Forschungen betreiben, mehr dazu in Kapitel 4.4.2.

4.2 Diskussion

Im Allgemeinen, kann gesagt werden, dass Objekte, welche unter besseren Lichtvoraussetzungen stehen auch entsprechend bessere Voraussagen über die Bounding Box und die zu extrahierende Textur treffen. Dies könnte daran liegen, dass die Ecken und Kanten der Objekte durch die Ausleuchtung klarer definiert sind. So ist neben dem Kontrast zwischen den einzelnen Objektseiten auch der Farbunterschied größer. Dies bietet dem neuronalen Netz mehr Möglichkeiten der Orientierung und Vergleiche.

Ein weiterer Diskussionspunkt ist die Farbe des Objekts. Gegen die Erwartung führen bunte Legobausteine zu besseren Ergebnissen als einfarbige Zusammensetzungen. Je nachdem wie

viele Details innerhalb einer Textur sind, sollte sich die Qualität der Textur verändern. Falls eine ganze Seite aus einer Farbe besteht, so sollte die Bestimmung der Bounding Box leichter fallen, da dies die Möglichkeit von Rauschen oder inkorrektur Klassifizierung verringern sollte. Tatsächlich trainiert das neuronale Netz besser auf dieser Art von Texturen, da es mehr Möglichkeiten der Kantenerkennung gibt. Die Menge der Testdatensätze lässt aber keine endgültige Bewertung zu, da dafür weitere Forschung nötig wäre (Kapitel 4.4.2). So könnte man andere Modelle mit denselben Datensätzen testen, um zu sehen, ob sich die Texturextraktion signifikant verändert. Ebenfalls wäre es interessant, mehr Testdatensätze zu erstellen, die zwar mehr Variation, aber ähnliche Beeinträchtigungen wie unterschiedliche Belichtungen zeigen.

Ein weiterer interessanter Punkt ist die prognostizierte Bounding Box vom vierten Legomix. Dieser ist in Form eines Dreiecks aufgebaut, ähnlich einer 2-dimensionalen Pyramide und besitzt daher in der Nähe der oberen Bounding Box Eckpunkte keinerlei Vertices. Überraschenderweise sagt das neuronale Netz jedoch auch dieses Objekt gut voraus. Dies zeigt, dass nicht nur die Position der Bounding Box Eckpunkte wichtig ist, sondern das Objekt als Ganzes in Betracht gezogen wird.

4.3 Theoretische Betrachtung

Die Extraktion einer Textur aus einem Objekt könnte in Bereichen wie Augmented Reality ein wichtiger Bestandteil werden. Interessant wäre die Idee, ob man mithilfe der Daten der 6D Pose Estimation die Qualität der Textur verbessern kann. Dies könnte mithilfe der Rotations- und Transformationsmatrix mathematisch hinterfragt werden.

Es ist schwierig, eine Verbindung zwischen einer 3×3 Matrix und einer größeren Textur aufzustellen. Man erhält zwar den Winkel und somit eine theoretische Grundlage, in welchem Winkel sich die zu entnehmende Textur zur Kamera befindet. Dies verbessert jedoch nur bedingt die Qualität der Textur. Eine Möglichkeit wäre es, den Wert des Winkels zu nutzen, um die Pixel je nach Winkel breiter zu ziehen. So könnte man berechnen, dass eine Textur sehr schräg zur Kamera positioniert ist und dann mithilfe der Matrizen den Winkel des Objekts zur Kamera berechnen. So ließe sich die Verzerrung der Textur errechnen. Diese weitere mathematische Betrachtung hat den Umfang der Arbeit überschritten.

Segment Anything ist ein Ansatz zur Segmentierung von Objekten in Bildern. Hierbei werden nicht nur vordefinierte Objekte, sondern alles, was auf einem Bild erscheint, segmentiert. Ein Vorteil dieses Ansatzes ist, dass quasi perfekte Masken von Objekten generiert werden

können. Dadurch würde die Textur-Extraktion verbessert, da diese eine exakte geometrische Abbildung des Objekts ermöglichen und somit eine eindeutige Zuordnung der Textur ermöglicht werden kann. Durch die Verwendung von Meta-Learning-Methoden kann der Ansatz zudem an neuen Bildern angepasst werden, um die Segmentierungsgenauigkeit weiter zu erhöhen. Auch wenn die Textur sehr präzise extrahiert werden könnte, würden keinerlei Informationen über die 3-dimensionale Lage des Objekts generiert und somit nicht der Winkel, in welchem sich die Textur in Relation zur Kamera befindet, berechnet werden.

Nvidias Bundle SDF 3D Objekt Generierung ist ein Verfahren zur Generierung von 3D-Objekten aus einer Sammlung von 2D-Bildern. Dabei wird ein implizites Modell des Objekts generiert, das als SDF (Signed Distance Function) dargestellt wird. Dieses Modell kann anschließend zur Generierung von 3D-Objekten genutzt werden. Der Vorteil dieses Ansatzes liegt darin, dass er unabhängig von der Textur der Objekte ist. Dadurch können auch Objekte generiert werden, die keine oder eine sehr geringe Textur aufweisen. Diese Eigenschaft macht den Ansatz besonders interessant für Texturextraktion, da Texturinformationen nicht immer verfügbar oder aussagekräftig sind.

4.4.1 Limitation des Modells

Aus den Daten des Modells, die aus der automatischen Extraktion der Textur hervorgegangen sind, konnten keine signifikanten Vorteile, die auf dem Fokus der vier Eckpunkte lagen, festgestellt werden. Die Tendenz wirkt sogar eher leicht negativ, wenn auch nicht in allen Bereichen. Dies lässt eventuell darauf schließen, dass die Abwandlung des Neuornalen Netztes nicht intensiv genug war und weiterhin ausbaufähig ist. Zudem ist das Methodik des Single-shotpose Modells für die Aufgabe der Texturextraktion nicht optimiert. Weitere Forschungen, wie die Inbetrachtnahme anderer 6D Pose Estimation Modelle, eine Verbindung zwischen den Textur Outputs, Nutzung neuerer Segmentations Ideen oder einer besseren Trainingsumgebung, deren Ideen jedoch über den Rahmen dieser Arbeit hinausgehen würden, wurden nicht erfasst. Zudem sind die Größen der extrahierten Texturen sowie die Auswahl der Testobjekte nicht ausreichend, um genaue Vorhersagen treffen zu können.

4.4.2 Implikationen

Aufgrund der vergleichsweise kleinen Datenmenge wird vorgeschlagen, weitere Studien mit mehr Daten durchzuführen. Wie schon in der Diskussion angedeutet, könnte es hilfreich sein, modernere 6D Pose Estimation Ansetze zu nutzen. Zudem wäre es nützlich, weitere Faktoren zur Bounding Box miteinzubeziehen. So könnte man einen pixelweisen Vergleich der prognostizierten Textur mit der Textur der Ground Truth schaffen, um ein exaktes Abbild im Training zu generieren. Auch könnte es sinnvoll sein, neben dem Trainingsdatensatz auch die Batch Size zu erhöhen.

4.5 Ausblick

6D Pose Estimation ist ein komplexes Thema, da es eine Vielzahl von Faktoren berücksichtigt, die zur genauen Extraktion von Texturen aus einem Objekt notwendig sind. Die automatische Texturextraktion mittels 6D Pose Estimation erfordert die Anwendung von verschiedenen Technologien wie Machine Learning, Robotik, Augmented Reality oder der Automobilindustrie. In dieser Studie wurde versucht, diese Technologien zu optimieren und zu verbessern, um eine genauere Extraktion von Texturen zu ermöglichen. Hierbei müssen verschiedene Faktoren wie Beleuchtung, Farben, Textur und Form des Objekts berücksichtigt werden.

Ein möglicher Ausblick für die Zukunft ist, dass die automatische Texturextraktion in Branchen vermehrt eingesetzt werden könnte, wie zum Beispiel in der Medizin, der Architektur und der Industrie. Die Technologie könnte dazu beitragen, die Effizienz und Genauigkeit von Arbeitsprozessen zu steigern und somit Zeit zu sparen, aber auch die Qualität zu verbessern. So könnten auch automatische Qualitätskontrollen von Produkten ermöglicht werden, um automatisch Fehler und Mängel auf der Oberfläche von Produkten zu erkennen. Darüber hinaus gibt es auch ethische und moralische Aspekte, die berücksichtigt werden müssen. Diese Technologie kann auch zur Überwachung und Kontrolle eingesetzt werden, was einige Bedenken hervorrufen kann. Daher ist es wichtig, dass die Technologie verantwortungsvoll eingesetzt wird und Freiheits- und Menschenrechte durch die Gesetzgebung geschützt werden.

Insgesamt ist die 6D Pose Estimation ein vielversprechendes Forschungsgebiet mit vielen Anwendungsmöglichkeiten. Es wird erwartet, dass weitere Entwicklungen in diesem Bereich dazu beitragen werden, die Technologie zu verbessern und neue Anwendungen zu ermöglichen.

5. Literaturverzeichnis

[Lev44]	K. Levenberg (1944) A Method for the Solution of Certain Problems in Least Squares Quarterly of Applied Mathematics S. 164 - 168
[FM80]	F. M. Andrews and A.C. McKennell (1980): Measures of self-reported well-being: their affective, cognitive, and other components https://link.springer.com/article/10.1007/BF00286474
[FB81]	M.A. Fischler and R.C. Bolles (1981) Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. https://dl.acm.org/doi/10.1145/358669.358692
[TZ96]	P. Torr and A. Zisserman (1996) MLESAC: A new robust estimator with application to estimating image geometry. https://www.robots.ox.ac.uk/~vgg/publications/2000/Torr00/torr00.pdf
[TD00]	P. Torr and C. Davidson (2000) IMPSAC: A synthesis of importance sampling and random sample consensus to effect multi-scale image matching for small and wide baselines. https://link.springer.com/content/pdf/10.1007/3-540-45053-X_52.pdf
[Der10]	Konstantinos G. Derpanis (2010) Overview of the RANSAC Algorithm http://www.cse.yorku.ca/~kosta/CompVis_Notes/ransac.pdf
[HW11]	Nikolaus Hotton, Paul Walter (2011) Ausgewählte Prozessbetrachtung von der Aufnahme bis zur Rezeption

	https://www.academia.edu/84450452/Stereoskope_HD_Produktion_Ausgew%C3%A4hlte_Prozessbetrachtungen_von_der_Aufnahme_bis_zur_Rezeption
[HCC+11]	S. Hinterstoisser, S. Holzer, C. Cagniart, S. Ilic, K. Konolige, N. Navab, and V. Lepetit (2011) Multimodal templates for real-time detection of texture-less objects in heavily cluttered scenes https://campar.in.tum.de/pub/hinterstoisser2011linemod/hinterstoisser2011linemod.pdf
[SSHB12]	Jörg Stückler, Ricarda Steffens, Dirk Holz, Sven Behnke (2012) Efficient 3D Object Perception and Grasp Planning for Mobile Manipulation in Domestic Environments https://www.academia.edu/22921962/Efficient_3D_object_perception_and_grasp_planning_for_mobile_manipulation_in_domestic_environments
[HLI+12]	Stefan Hinterstoisser, Vincent Lepetit, Slobodan Ilic, Stefan Holzer, Gary Bradski, Kurz Konolige, Nassir Navan (2012) Model Based Training, Detection and Pose Estimation of Texture- Less 3D Objects in Heavily Cluttered Scenes http://www.stefan-hinterstoisser.com/papers/hinterstoisser2012accv.pdf
[DKB14]	Michael Donoser, Peter Kotschieder, Horst Bischof (2014) Robust Planar Target Tracking and Pose Estimation from Single Concavity https://www.yumpu.com/en/document/view/26456272/robust-planar-target-tracking-and-pose-estimation-from-a-single-
[CTL+15]	T.J.Chong, X.J. Tang, C.H. Leng, M.Yogeswaran, O.E.Ng, Y.Z. Chong (2015) Sensor Technologies and Simultaneous Localization and Mapping (SLAM)

	https://www.researchgate.net/publication/290055590_Sensor_Technologies_and_Simultaneous_Localization_and_Mapping_SLAM
[Bos16]	Boston Dynamics (2016) Atlas, the next generation https://www.youtube.com/watch?v=rVlhMGQgDkY
[RDGF16]	Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi (2016) You Only Look Once: Unified, Real-Time Object Detection https://arxiv.org/abs/1506.02640v5
[RHGS16]	Shaoqing Ren, Kaiming He, Ross Girshick, Jian Sun (2016) Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks https://arxiv.org/abs/1506.01497v3
[RF16]	J. Redmon and A. Farhadi (2016) YOLO 9000: Better, Faster, Stronger https://arxiv.org/abs/1612.08242
[QSMG16]	Charles R. Qi, Hao Su, Kaichun Mo, Leonidas J. Guibas (2016) PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation https://arxiv.org/abs/1612.00593
[KMT+17]	W. Kehl, F. Manhardt, F. Tombari, S. Ilic, and N. Navab (2017) SSD-6d: Making RGB-based 3d detection and 6d pose estimation great again https://arxiv.org/abs/1711.10006
[RL17]	M. Rad and V. Lepetit (2017) BB8: A scalable, accurate, robust to partial occlusion method for predicting the 3D poses of challenging objects without using depth,

	http://arxiv.org/abs/1703.10896v2
[HHO+17]	Tomas Hodan, Pavel Haluza, Stepan Obdrzalek, Jiri Matas, Manolis Lourakis, Xenophon Zabulis (2017) T-LESS: An RGB-D Dataset for 6D Pose Estimation of Texture-less Objects https://arxiv.org/abs/1701.05498
[TSF18]	B. Tekin, S. N. Sinha, and P. Fua, (2018) Real-time seamless single shot 6d object pose prediction http://arxiv.org/abs/1711.08848v4
[PLH+19]	Sida Peng, Yu.An Liu, Qi-Xing Huang, Hujun Bao, Xiaowei Zhou (2019) PVNet: Pixel-wise Voting Network for 6DoF Pose Estimation https://zju-3dv.github.io/pvnet/
[JYX+21]	Chao Jia, Yinfei Yang, Ye Xia, Yi-Ting Chen, Zarana Parekh, Hieu Pham, Quoc V. Le, Yunhsuan Sung, Zhen li, Tom Duerig (2021) Scaling Up Visual and Vision-Language Representation Learning With Noisy Text Supervision https://arxiv.org/abs/2102.05918
[RPG+21]	Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, Ilya Sutskever (2021) Zero-Shot Text-to-Image Generation https://arxiv.org/abs/2102.12092
[Ivi22]	Ivisio GmbH Schematic of the reprojection error https://www.camcalib.io/post/what-is-the-reprojection-error

[Bai22]	Hao Bai (2022) ICP Algorithm: Theory, Practice And Ist SLAM-oriented Taxonomy https://arxiv.org/abs/2206.06435
[RFM23]	Mariano Rodriguez, Gabriele Facciolo, Jean-Michel Morel (2023) Robust Homography Estimation from Local Affine Maps https://doi.org/10.5201/ipol.2023.356
[DWS+23]	Maximilian Denniger, Dominik Winkelbauer, Martin Sundermeyer, Wout Boerdijk, Markus Knauer, Klaus H. Strobl., Matthias Humt, Rudolph Triebel (2023) BlenderProc2: A Procedural Pipeline for Photorealistic Rendering https://doi.org/10.21105/joss.04901
[KMR+23]	Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Aleaxander C. Berg, Wan-Yen Lo, Piotr Dollar, Ross Girshick (2023) Segment Anything https://github.com/facebookresearch/segment-anything
[WTB+23]	Bowen Wen, Jonathan Tremblay, Valts Blukis, Stephen Tyree, Thomas Müller, Alex Evans, Dieter Fox, Jan Kautz, Stan Birchfield BundleSDF: Neural 6-DoF Tracking and 3D Reconstruction of Unknown Objects https://arxiv.org/abs/2303.14158
[Tom23]	Nikhil Tomar A diagram representing the Intersection over Union (IoU) https://idiotdeveloper.com/what-is-intersection-over-union-iou/

6. Zusätzliches Material

Der in dieser Arbeit benutzte Code ist verfügbar unter:

<https://github.com/deweezy12/6dposeest>

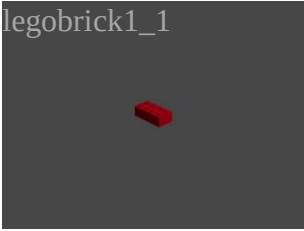
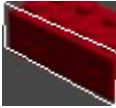
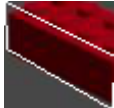
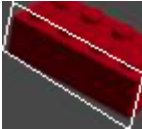
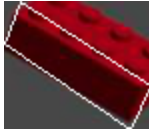
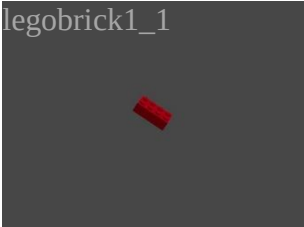
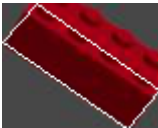
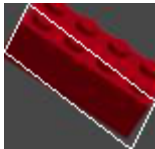
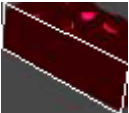
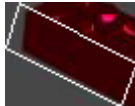
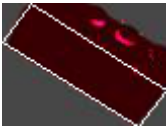
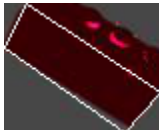
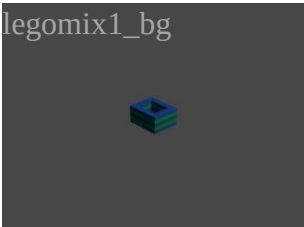
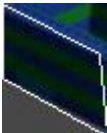
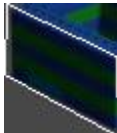
```
max_distance_inside = 4.0 # maximale Pixel nach innen
max_distance_outside = 6.0 # maximale Pixel nach außen
weight = 0.25

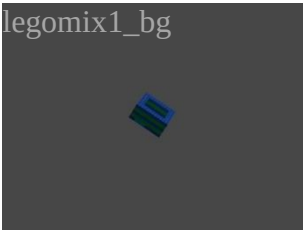
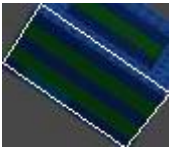
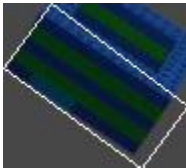
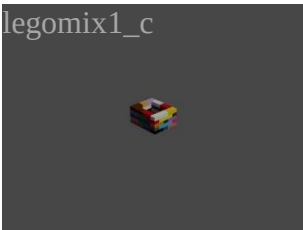
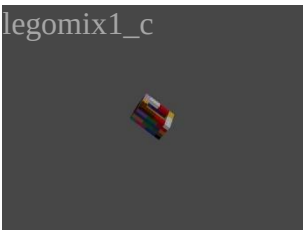
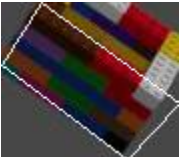
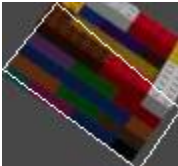
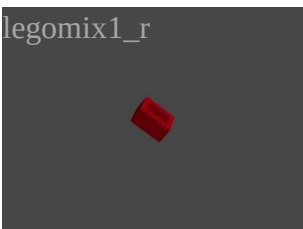
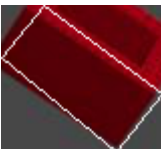
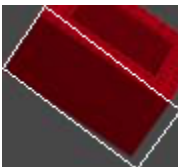
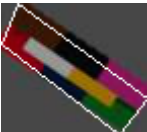
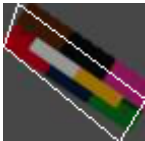
for i in range(points_pr.shape[0]):
    distance = math.sqrt((points_gt[i,0] - points_pr[i,0])**2 + (points_gt[i,1] - points_pr[i,1])**2)
    if (min_x <= points_pr[0][0] <= max_x and min_x <= points_pr[0][1] <= max_x) and (min_y <=
points_pr[1][0] <= max_y and min_y <= points_pr[1][1] <= max_y):
        error = min(distance ** 2, max_distance_inside)
        err_percentage = (error / max_distance_inside) * weight
    else:
        error = min(distance, max_distance_outside)
        err_percentage = (error / max_distance_outside) * weight
    total_error += err_percentage

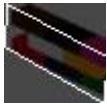
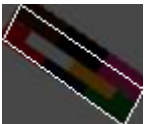
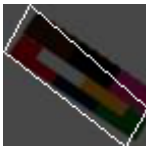
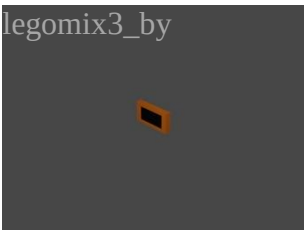
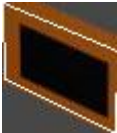
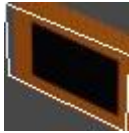
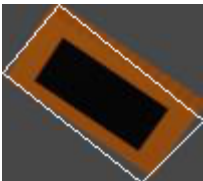
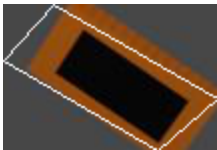
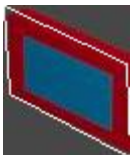
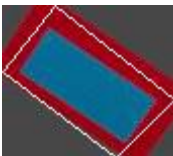
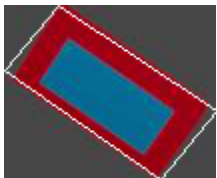
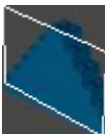
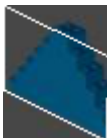
correctness = 1.0 - total_error
```

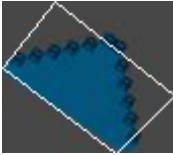
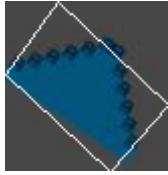
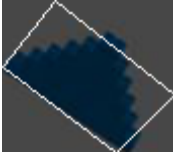
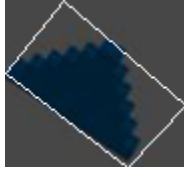
3 Programmcode (Bewertungsmetrik, Die Variablen definieren die maximalen Grenzen für eine Bewertung, die if Funktion entscheidet, ob sich ein Punkt innerhalb oder außerhalb der Textur befindet. Points_pr sind die vorhergesagten Punkte)

Hier sind einige Texturextraktionen als Ergebnisse in Bildern

Original JPEG	9 Keypoints Texture	4 Keypoints Texture
legobrick1_1 		
legobrick1_1 		
legobrick1_1 		
legobrick1_2 		
legobrick1_2 		
legomix1_bg 		

legomix1_bg			
legomix1_c			
legomix1_c			
legomix1_r			
legomix1_r			
legomix2_1			
legomix2_1			

legomix2_2 		
legomix2_2 		
legomix3_by 		
legomix3_by 		
legomix3_rb 		
legomix3_rb 		
legomix4_1 		

legomix4_1 		
legomix4_2 		
legomix4_2 		

Name, Vorname:

Erklärung

gem. § 15 Abs. 6 PO (Allgemeine Bestimmungen)

Hiermit erkläre ich, dass ich die von mir eingereichte Abschlussarbeit (Bachelor-Thesis) selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie Stellen der Abschlussarbeit, die anderen Werken dem Wortlaut oder Sinn nach entnommen wurden, in jedem Fall unter Angabe der Quelle als Entlehnung kenntlich gemacht habe.

.....
Datum

.....
Unterschrift

Erklärung

Hiermit erkläre ich mich damit einverstanden, dass meine Abschlussarbeit (Bachelor-Thesis) wissenschaftlich interessierten Personen oder Institutionen und im Rahmen von externen Qualitätssicherungsmaßnahmen des Studienganges zur Einsichtnahme zur Verfügung gestellt werden kann.

Korrektur- oder Bewertungshinweise in meiner Arbeit dürfen nicht zitiert werden.

.....
Datum

.....
Unterschrift